

Actividad 7 Proyecto Final: API REST

Valdez Miranda Elías

Ingeniería en Sistemas de Información, Universidad de Sonora

Clave 4122: Desarrollo de Sistemas IV

Profesor Meza Ibarra Iván Dostoyewski

12 de mayo de 2026



Índice

Introducción...	1
Tema del proyecto...	2
Descripción del sistema...	3
Base de datos...	10
Web app...	13
API REST...	19
Pruebas...	26
Conclusión...	59
Fuentes de información...	60

Introducción

En la planificación e implementación de soluciones para diversos problemas, se pueden encontrar diferentes formas de llegar a las mismas metas esperadas. Esto persiste y es especialmente relevante en el desarrollo de sistemas que solucionen ciertas problemáticas. Se pueden encontrar muchas y diferentes soluciones a un solo problema, lo que dificulta la elección de una como la definitiva. Por esto, es importante reconocer las ventajas y desventajas que presenta cada solución.

En el curso, hemos trabajado con diferentes formas de manejar la información utilizando Python para el diseño de APIs REST con Flask, mostrando la versatilidad de implementar rutas, códigos de estatus y estructuras de datos como JSON para crear sistemas simples, flexibles y eficientes. Sin embargo, no es la única forma de manejar la información, pues presenta ciertas complicaciones al interactuar con usuarios no familiarizados con dichas estructuras o herramientas. Para esto, se empieza a manejar el uso de plantillas y las mismas rutas para el desarrollo de aplicaciones web, las que facilitan la interacción con el sistema.

A pesar de las diferencias entre las implementaciones de ambas formas de manejo de datos, es importante reconocer que su efectividad se ve mejorada en su uso conjunto, optando por ambas soluciones para diferentes partes del sistema.

Con este objetivo, esta práctica demostrará el desarrollo de un sistema para asistir en el manejo de información en una situación real y práctica, observando las ventajas, desventajas, facilidades y complicaciones en la implementación de la solución con Python y Flask.

Tema del proyecto

El tema de este proyecto parte como una evolución de una solución solicitada para la materia *Servidores II*, en la que, de manera similar, se solicitó el desarrollo de una aplicación web para la solución de una problemática real utilizando una base de datos. En este, identifique complicaciones en el manejo de órdenes dentro de una tienda de comida en la UNISON (Los compadres), desarrollando el siguiente proyecto como una solución:

Se elaborará una aplicación web para el manejo de pedidos de la tienda de alimentos Los Compadres en la Universidad de Sonora, identificando a los usuarios estos por medio de un sistema que almacene sus correos y contraseñas. Los clientes podrán observar productos disponibles en la tienda, así como colocar órdenes de estos, las que se podrán pagar en línea, con efectivo o por tarjeta. Los administradores también podrán interactuar con el sistema, con funciones adicionales, como el manejo de productos y órdenes.

Sin embargo, para los fines de este proyecto, se agrega el desarrollo de una API REST, la que permita manejar los datos de la base de datos en la que se almacena la información de los usuarios, las órdenes, los productos y los pagos. Con la problemática y la propuesta de solución, se ejemplifica un caso real que se podría desarrollar para facilitar el manejo de la información en una entidad, en este caso, la tienda “Los Compadres”.

Descripción del sistema

Para implementar la solución descrita en el problema anterior, se definen dos actores que podrán acceder a la aplicación. Aunque se consideraron roles adicionales, se busca mantener una escala reducida del proyecto, buscando un mayor énfasis en el manejo de la información y la implementación de las diferentes tareas como aplicación web y API REST. Dicho esto, se presentan los actores:

Cliente: Un cliente describe a aquellos usuarios a los que se les busca proveer el servicio. Podrán realizar y pagar pedidos, además de verificar el estado de sus órdenes. Se registrarán por medio de una pantalla de registro, utilizando un correo electrónico, un nombre de usuario y una contraseña. Es el rol que recibe un usuario por defecto.

Administrador: Un administrador describe a aquellos usuarios que operarán los servicios para la satisfacción del cliente. Cumplen con las funciones de gerentes del negocio y administradores de inventario, agrupadas en un rol genérico. Se registrarán de forma manual por medio de la base de datos, recomendando la creación de al menos un usuario con rol de administrador para la efectiva operación del sistema.

Las funciones de la aplicación web se definieron como requerimientos funcionales para los dos actores anteriores, desarrollados de la siguiente forma:

Identificación del requerimiento:	RF01
Nombre del requerimiento:	Inicio de sesión
Características:	Pantalla de bienvenida con formulario para inicio de sesión.

Descripción del requerimiento:	Al entrar en la interfaz inicial de la aplicación, los usuarios verán una pantalla de bienvenida de la tienda, junto con un formulario para el inicio de sesión, solicitando dos campos: • Nombre de usuario o correo electrónico • Contraseña Las entradas del usuario se validarán y redirigirán a este a su panel principal correspondiente. Si las credenciales proporcionadas son incorrectas, se redirigirá al usuario al panel de órdenes (RF03). En caso de no contar con un usuario, se ofrecerá un enlace para acceder a un formulario en el que se pueda crear uno nuevo (RF02).
Rol que ejecuta:	Cliente, Administrador
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF02
Nombre del requerimiento:	Registro de Cliente
Características:	Formulario para el registro de un nuevo usuario.
Descripción del requerimiento:	De no contar con un usuario, el cliente podrá acceder a un formulario desde la interfaz inicial de la aplicación (RF01), el que contará con los siguientes campos: • Nombre • Nombre de usuario • Correo electrónico • Contraseña • Confirmación de contraseña Cuando el usuario envíe las entradas, se validarán. Si las credenciales proporcionadas son inválidas (el nombre de usuario o el correo electrónico ya ha sido reclamado por otro usuario o exceden los límites establecidos por la base de datos), se le informará al usuario. Si las credenciales proporcionadas son válidas, se creará un nuevo cliente con los datos proporcionados. En dicho caso, se redirigirá al cliente al panel de órdenes (RF03). Por medio de esta interfaz solo se podrán registrar clientes.
Rol que ejecuta:	Cliente
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF03
Nombre del requerimiento:	Panel de órdenes
Características:	Visualización de órdenes por los clientes en línea
Descripción del requerimiento:	Pantalla principal del programa. Presentará una tabla con ordenes incompletas realizadas por los clientes. Para un cliente, esta vista proporcionará una tabla con información actualizada correspondiente a sus órdenes incompletas, así como su estado actual. Los clientes podrán modificar las órdenes incompletas. Si una orden cuenta con el estado “En carrito”, el cliente podrá escoger un método de pago y confirmar la orden. Para un administrador, esta vista proporcionará una tabla con información actualizada correspondiente a todas las órdenes incompletas, ordenadas por el tiempo de recolección programado. Contarán con la opción de modificar el estado de una orden seleccionada.
Rol que ejecuta:	Cliente, Administrador
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF04
Nombre del requerimiento:	Agregar orden
Características:	Formulario para la creación de una nueva orden de producto
Descripción del requerimiento:	El cliente podrá acceder a una sección en la que podrá visualizar por medio de una lista los productos ofrecidos por la tienda. La lista de productos deberá contener información actualizada, incluyendo su nombre, su precio y su disponibilidad. Si un producto observado está disponible, se podrá seleccionar. Junto a la lista de selección de productos, se contará con un campo que permita introducir la cantidad deseada de un producto. Si toda la información fue introducida de forma correcta, se almacenará la orden del cliente en memoria y se regresará al cliente al panel de órdenes (RF03). Si la información es inválida, se informará al usuario del error.
Rol que ejecuta:	Cliente
Prioridad del requerimiento: Alta	

Identificación del requerimiento:	RF05
Nombre del requerimiento:	Ver historial de órdenes
Características:	Tabla de visualización de órdenes pasadas de un cliente
Descripción del requerimiento:	El cliente podrá acceder a una tabla que muestre información relevante de órdenes solicitadas por este en el pasado. Las ordenes se presentarán sin importar su estado actual, ordenadas en orden cronológico.
Rol que ejecuta:	Cliente
Prioridad del requerimiento: Baja	

Identificación del requerimiento:	RF06
Nombre del requerimiento:	Ver productos
Características:	Tabla de visualización de productos ofrecidos en el sistema
Descripción del requerimiento:	Los administradores podrán visualizar la información actualizada de los productos almacenados en el sistema por medio de un tablero, en el que podrán modificar atributos como su precio y disponibilidad, además de eliminarlos de la lista de productos accesibles por el sistema.
Rol que ejecuta:	Administrador
Prioridad del requerimiento: Media	

Identificación del requerimiento:	RF07
Nombre del requerimiento:	Agregar Producto
Características:	Formulario de inserción de producto nuevo
Descripción del requerimiento:	Los administradores podrán acceder a un formulario en el que se podrá crear un nuevo producto por ofrecer utilizando los siguientes campos: • Nombre del producto • Precio por unidad • Disponibilidad Si se envía información válida y completa al sistema, se regresará al administrador a la venta de visualización de productos (RF06). Si la información

	es inválida, se informará al usuario del error y se restaurará el estado original de la ventana.
Rol que ejecuta:	Administrador
Prioridad del requerimiento: Media	

Se pueden reconocer ciertos problemas con el diseño del sistema, pero, reiterando la idea descrita anteriormente, se mantiene un nivel de abstracción y simplificación alto para concentrarse en la implementación del sistema para el manejo de datos, el objetivo principal de la práctica. Algunos de los problemas observados se desarrollan a continuación:

Falta de opciones para el manejo de usuarios: Se omite la elaboración de funciones de calidad para la modificación de información por parte de los usuarios en el sistema, incluyendo su nombre, nombre de usuario, correo electrónico y contraseña.

Simplificación de la relación orden: Originalmente, la orden comprendía una colección de productos, estos últimos almacenando la cantidad de existencias para facilitar la flexibilidad y elaboración de órdenes complejas utilizando el sistema. Sin embargo, reconociendo complicaciones en la implementación de un panel de órdenes con productos dinámicos, además de las verificaciones necesarias, decidí simplificar la orden a un producto singular, especificando la cantidad por solicitar de esto. Al limitar la cantidad de órdenes que un usuario puede colocar, esto también limita la capacidad de los usuarios en ordenar, limitación que se reconoce como un problema en un despliegue completo de la aplicación.

Estructura de la relación producto: En el sistema, un producto se identifica por medio de una ID, una clave primaria de identidad. En un sistema real, se utilizarían identificadores en formato EAN-13 para posibilitar el uso de lectores de barras y trabajar de forma más correcta con los productos. Esta discrepancia presenta un considerable obstáculo en la identificación del

producto, problema exacerbado por la necesidad de trabajar con proveedores utilizando tales identificadores.

Limitaciones en métodos de pago: La relación *Pago* es simplificada a ser un registro de una transacción monetaria entre el cliente y el sistema, sea en persona, en línea o por tarjeta. Por limitaciones en la implementación del sistema, el pago no almacena más información que el monto por pagar y el monto pagado, además del método de pago. Registros de transacciones más completos incluirían detalles adicionales, como una selección de dígitos de la tarjeta utilizada (en caso de usarla), detalles de facturación, entre otros.

Sobre simplificación de las relaciones de registros: Las relaciones de registros se simplificaron a incluir únicamente una referencia a la entrada relevante, un comentario genérico sobre este y la fecha del registro, lo que omite detalles en caso de insertar o actualizar una entrada. Similar a cambios anteriores, se realizó para facilitar el trabajo con la base de datos y enfocarse en el desarrollo del servidor.

Por el otro lado, la API REST desarrollada toma una perspectiva más directa: Se ofrece funcionalidad CRUD completa a las cuatro entidades principales del programa (descritas en la siguiente sección). No obstante, para preservar ciertas secciones de la funcionalidad del programa, se agregan rutas adicionales para un manejo más apropiadas de ciertas entidades.

El proyecto cuenta con los siguientes directorios y archivos para su funcionamiento:

app: Almacena los archivos con la lógica y estructura de la aplicación

- **api:** Almacena los archivos de lógica para la API REST

- **static:** Almacena los archivos estáticos del proyecto (aquellos que no se modifican)
- **templates:** Almacena las plantillas HTML del proyecto, modificadas por Flask para la correcta presentación de la web app
- **web:** Almacena los archivos de lógica para la web app
- **db.py:** Almacena las funciones de acceso a la base de datos por parte de la web app
- **db_api.py:** Almacena las funciones de acceso a la base de datos por parte de la API REST
- **helpers.py:** Almacena funciones de asistencia para el manejo de la información en la web app y API REST

docs: Almacena la documentación del proyecto, incluyendo el archivo de definición de la base de datos y el documento actual.

.env: Almacena las credenciales de acceso a la base de datos

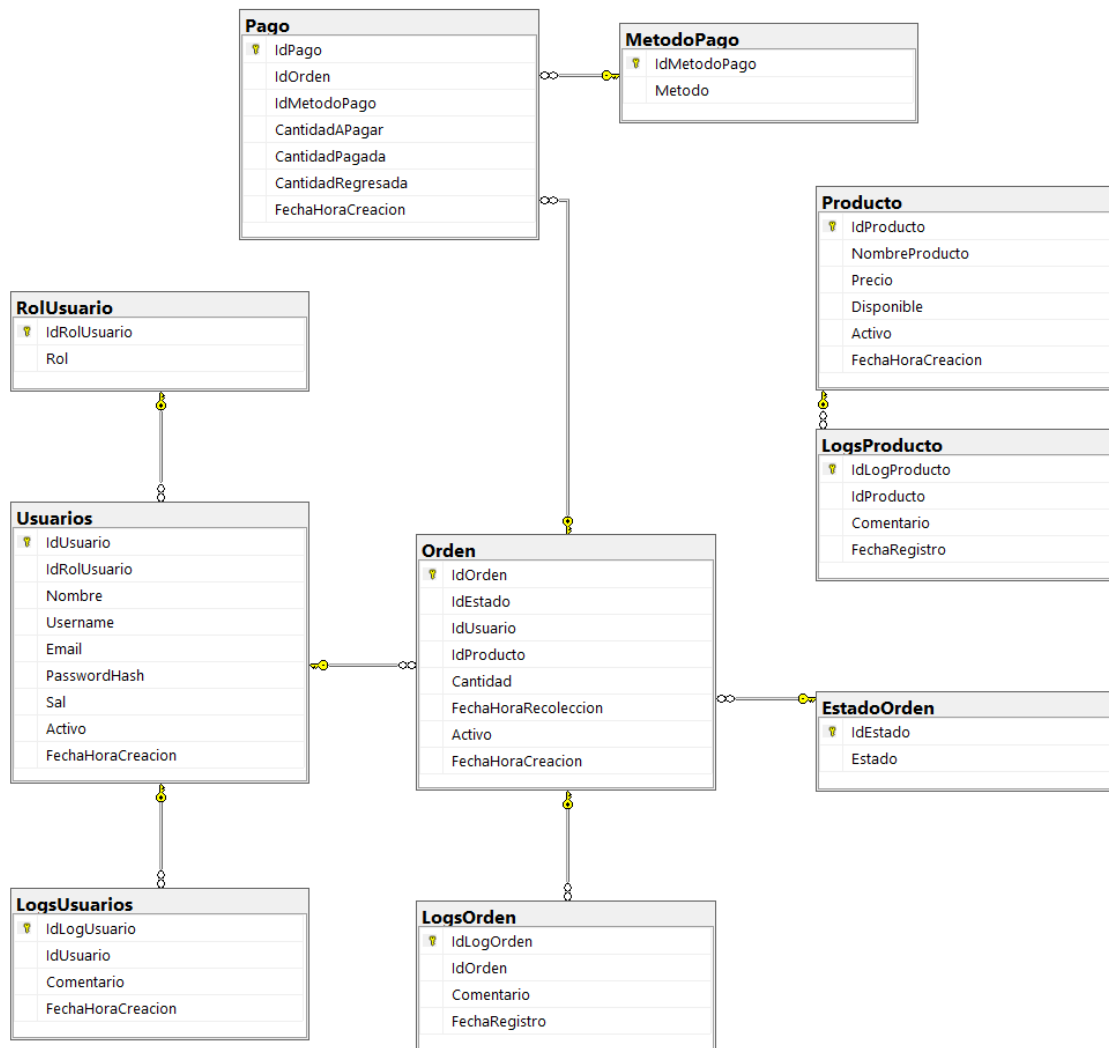
run.py: Archivo de arranque del programa. Inicia la web app y la API REST.

El diseño del programa implementa blueprints, los que permiten modularizar la aplicación web y la API REST en dos rutas separadas, facilitando el acceso a ambas partes del sistema. Este punto se desarrollará con más detalles en sus respectivas secciones.

Base de datos

Por el uso de Windows Server en la práctica original de *Servidores II*, se utiliza Microsoft SQL Server como la base de datos del sistema, ofreciendo como ventaja la posibilidad de utilizar procedimientos almacenados, funciones y otras características más avanzadas con la desventaja de requerir una mayor configuración previo a su uso.

La base de datos cuenta con la siguiente estructura:



Las relaciones se pueden dividir en tres categorías:

Relaciones de atributo: Las relaciones de atributo almacenan únicamente una clave primaria y una descripción en formato NVARCHAR, utilizando las entradas de estas relaciones como referencia para atributos de otras, necesario para una mayor normalización de las relaciones. Las claves primarias no se registran como identidad, y, por el contrario, requiere la definición manual de las IDs, lo que facilita la definición e identificación de entradas específicas. Estas tablas no se buscan modificar, por lo que se excluyen de las operaciones CRUD. En el sistema se pueden observar tres relaciones bajo esta categoría:

- RolUsuario
- MetodoPago
- EstadoOrden

Relaciones generales: Las relaciones generales almacenan información relevante a los usuarios en el sistema. Contienen una colección de atributos relevantes a cada relación, identificados por una clave primaria de identidad. Para mantener la integridad referencial de la base de datos en caso de actualizar o eliminar las entradas en las relaciones generales, se utiliza el atributo Activo para identificar aquellas entradas que son relevantes al sistema, sustituyendo ciertas operaciones de actualización o eliminación por el cambio de esta variable para bloquear cambios posteriores a entradas relevantes utilizando triggers (por ejemplo, evitar la eliminación o modificación de órdenes completas). Tres relaciones entran en esta categoría;

- Usuarios
- Orden
- Producto

Relaciones de registros: Las relaciones de registros, como su nombre indica, se encargan de almacenar registros simples de operaciones sobre las relaciones generales en la base de datos. Utilizan una clave primaria de identidad, un comentario en formato NVARCHAR, una fecha de registro en formato DATETIME y una clave foránea a la entrada relevante. La relación pago es una excepción a la descripción anterior, pues presenta una colección de atributos expandida para registrar información relevante de una transacción en el sistema. De manera similar a las relaciones de atributo, estas tablas no se buscan modificar, por lo que se excluyen de las operaciones CRUD, con excepción de la tabla Pago, la que, por su importancia en el manejo de una orden, requiere modificarse en ciertos momentos. Cuatro relaciones pertenecen a esta categoría:

- LogsUsuarios
- LogsOrden
- LogsProducto
- Pago

La implementación completa de la base de datos se puede encontrar en la sección de documentos del proyecto con nombre “db_script.sql”.

Web App

La lógica relacionada a la aplicación web se encuentra en la carpeta “web” dentro de la carpeta “app” en la raíz del proyecto. La ruta base del blueprint se almacena en “/” Se pueden observar dos archivos dentro de esta:

__init__.py: Almacena instrucciones ejecutadas al iniciar la aplicación de Flask, encargadas de definir el blueprint e importar las funciones de las rutas correspondientes a la aplicación web. El archivo no cuenta con lógica fuera de las importaciones, por lo que no es necesario entrar en mayor profundidad respecto a su funcionamiento.

routes.py: Almacena las rutas relevantes a la aplicación web. Dentro de estas se almacena la lógica completa del manejo de páginas, formularios y la información recibida para su tratamiento con la base de datos. A continuación, se describe el funcionamiento de las rutas.

- **/login [GET, POST]:** Maneja el inicio de sesión del usuario. Al solicitarse con el método GET, renderiza una plantilla con una página de bienvenida del sistema, mostrando fotos de la ubicación, ofreciendo, además, dos campos para introducir un nombre de usuario o correo electrónico y la contraseña del usuario registrado. Al solicitarse con el método POST, se buscan las entradas del usuario y se validan sus credenciales con la base de datos. Si no se encuentran los datos o se falla la verificación de las credenciales, se le informa al usuario con un error. De lo contrario, se le redirige a la ruta principal del programa (“/”) después de almacenar la ID del usuario y su rol en el caché de la sesión.

- **/logout [GET]:** Cierra la sesión del usuario activo. Soporta únicamente el método GET, limpiando la sesión y redirigiendo a la ruta principal del programa (“/”).
- **/register [GET, POST]:** Maneja el registro de nuevos usuarios. Al solicitarse con el método GET, renderiza una plantilla con campos para el registro de un nuevo usuario, incluyendo nombre completo, nombre de usuario, correo electrónico, contraseña y confirmación de la contraseña. Al solicitarse con el método POST, se buscan las entradas del usuario y se validan sus credenciales con la base de datos. Si no se encuentran los datos, se encuentran discrepancias entre las contraseñas introducidas o ya existe un usuario con el nombre de usuario/correo electrónico ingresado, se le informa al usuario con un error. De lo contrario, se registra al usuario en la base de datos, generando un hash de contraseña segura, almacenando su Id y rol de usuario (cliente) en la sesión del navegador y devolviéndolo a la ruta principal del programa. Por medio de esta interfaz solo se pueden registrar clientes.
- **/(ruta principal) [GET]:** Página principal de la aplicación. Utiliza el decorador @login_required para trabajar con la información de un usuario específico. Al solicitarse con el método GET, renderiza una plantilla con un tablero de órdenes específico el tipo de usuario. Si el rol de usuario de la sesión es un cliente, se obtienen las órdenes incompletas específicas de este, mientras que, si el rol de usuario de la sesión es un administrador, se obtienen todas las órdenes incompletas del sistema, devolviendo un error si no se identifica el rol de usuario de la sesión. Por medio de esta ruta se provee acceso al resto de rutas de administración de órdenes descritos en apartados posteriores.

- **/producto/ver [GET]:** Muestra el catálogo de productos registrados en el sistema. Utiliza el decorador `@login_required` para trabajar con la información de un usuario específico. Al solicitarse con el método GET, renderiza una plantilla con un tablero mostrando los productos almacenados en la base de datos, permitiendo su modificación. El acceso de esta ruta se permite únicamente a los administradores, devolviendo un error a usuarios no autorizados.
- **/producto/agregar [GET, POST]:** Maneja el registro de nuevos productos. Utiliza el decorador `@login_required` para trabajar con la información de un usuario específico. Al solicitarse con el método GET, renderiza una plantilla con campos para el registro de un nuevo producto, incluyendo nombre de producto, precio y una indicación de la disponibilidad del producto. Al solicitarse con el método POST, se buscan las entradas del usuario y se validan previo a su inserción. Si no se encuentran los datos o se encuentran inconsistencias en la lógica (precio negativo), se le informa al usuario con un error. De lo contrario, se registra el producto en la base de datos y se devuelve a la ruta de visualización de productos (`/producto/ver`). El acceso de esta ruta se permite únicamente a los administradores, devolviendo un error a usuarios no autorizados.
- **/producto/editar/<int:id_producto> [POST]:** Actualiza la información de un producto existente. Utiliza el decorador `@login_required` para trabajar con la información de un usuario específico. Esta ruta soporta únicamente el método POST, recibiendo información actualizada sobre el nombre, precio y disponibilidad del producto por actualizar. Si no se encuentran los valores de actualización o se encuentran errores de lógica, se le informa al usuario con un

error. De lo contrario, se actualiza el producto en la base de datos y se devuelve a la ruta de visualización de productos (/producto/ver). El acceso de esta ruta se permite únicamente a los administradores, devolviendo un error a usuarios no autorizados.

- **/producto/eliminar/<int:id_producto> [POST]:** Elimina un producto del sistema. Utiliza el decorador @login_required para trabajar con la información de un usuario específico. Esta ruta soporta únicamente el método POST, tomando la Id del producto proporcionada en la ruta para eliminar un producto identificado de la base de datos. El acceso de esta ruta se permite únicamente a los administradores, devolviendo un error a usuarios no autorizados.
- **/orden/agregar [GET, POST]:** Maneja la creación de una nueva orden por parte de un usuario. Utiliza el decorador @login_required para trabajar con la información de un usuario específico. Al solicitarse con el método GET, renderiza una plantilla con campos para la creación de una nueva orden incluyendo un campo con todos los productos disponibles (obtenidos desde la base de datos), la cantidad de unidades del producto deseadas, la fecha de recolección programada y la hora de recolección programada. Al solicitarse con el método POST, se buscan las entradas del usuario y se validan previo a su inserción. Si no se encuentran los datos o se encuentran inconsistencias en la lógica (cantidad negativa o igual a 0), se le informa al usuario con un error. De lo contrario, se registra la orden en la base de datos con Id de estado 1 (en carrito) y se devuelve a la ruta principal del programa (/).

- **/orden/editar/<int:id_orden> [POST]:** Actualiza los valores permitidos (cantidad) de una orden en el carrito. Utiliza el decorador `@login_required` para trabajar con la información de un usuario específico. Esta ruta soporta únicamente el método POST, tomando la Id del producto proporcionada en la ruta y la cantidad introducida en un formulario para actualizar la información de la orden dada. Si se falla en encontrar los valores del formulario o se encuentra un error en la lógica (cantidad negativa o igual a 0), se informa al usuario con un error. De lo contrario, se actualizan los valores de la orden con los nuevos recibidos.
- **/orden/confirmar/<int:id_orden> [POST]:** Confirma una orden en el carrito y registra el pago inicial. Utiliza el decorador `@login_required` para trabajar con la información de un usuario específico. Esta ruta soporta únicamente el método POST. En la plantilla previa al acceso de este enlace, se solicita la inserción del método de pago, lo que define la forma en la que se creará el registro del pago. Si el método de pago es por efectivo o tarjeta (físico), se registra un pago incompleto, almacenando únicamente la id de la orden, el método de pago y la cantidad a pagar. Por el otro lado, si el método de pago es en línea, se registra un pago completo, incluyendo los campos anteriores además de la cantidad pagada y la cantidad regresada (\$0, pues el pago en línea es exacto).
- **/orden/completar/<int:id_orden> [POST]:** Completa una orden confirmada y actualiza la información de pago si es pertinente. Utiliza el decorador `@login_required` para trabajar con la información de un usuario específico. Esta ruta soporta únicamente el método POST. En la plantilla previa al acceso de este enlace, se ofrecen alternativas de acuerdo con el método de pago registrado de la

orden: Si el método de pago es por efectivo, se toma como entrada la cantidad de dinero con la que pagó el usuario y se valida que el dinero introducido sea mayor o igual al precio total de la orden. Si la entrada es inválida, se presenta un error al usuario. Si el método de pago es por tarjeta, se le solicita al usuario introducir una tarjeta y seguir las instrucciones de la terminal. Al ser solo una prueba del concepto, no se solicitan confirmaciones adicionales, y, al confirmar la acción, se valida automáticamente la orden. Si el método de pago es en línea, se reitera que la orden ha sido pagada y no se hacen verificaciones adicionales. En los tres casos, si no se encuentran errores, se actualiza el estado de la orden a “Completada” y se marca como inactiva, evitando su modificación/eliminación de la base de datos, además de actualizar el pago para las órdenes pagadas en efectivo y por tarjeta. Al terminar, se regresa al usuario a la ruta principal del programa (/). El acceso de esta ruta se permite únicamente a los administradores, devolviendo un error a usuarios no autorizados.

- **/orden /eliminar/<int:id_orden> [POST]:** Elimina una orden del sistema. Utiliza el decorador `@login_required` para trabajar con la información de un usuario específico. Esta ruta soporta únicamente el método POST, tomando la Id de la orden proporcionada en la ruta para eliminar una orden identificada en la base de datos. Los usuarios y los administradores podrán acceder a esta interfaz, pues ambos deberán tener la autonomía de eliminar órdenes incompletas.
- **/historial [GET]:** Muestra el historial de órdenes completas de un cliente. Al solicitarse con el método GET, renderiza un tablero con todas las órdenes realizadas por el cliente en el pasado ordenadas por la fecha de creación de la

orden, sin importar su estado. Esto permite ver órdenes completadas y canceladas, a diferencia del tablero en la ruta principal del programa (/) .El acceso de esta ruta se permite únicamente a los clientes, devolviendo un error a los usuarios no autorizados.

API REST

La lógica relacionada a la API REST se encuentra en la carpeta “api” dentro de la carpeta “app” en la raíz del proyecto. Al ser la API una herramienta orientada a desarrolladores y a los administradores del sistema, se omite el uso de validaciones para manejar la información, ofreciéndola directamente a cada consulta, una consideración que facilitó considerablemente la implementación, pero hace al sistema más inseguro. Se pueden observar dos archivos dentro de la carpeta app:

__init__.py: Almacena instrucciones ejecutadas al iniciar la aplicación de Flask, encargadas de definir el blueprint e importar las funciones de las rutas correspondientes a la API REST. El archivo no cuenta con lógica fuera de las importaciones, por lo que no es necesario entrar en mayor profundidad respecto a su funcionamiento.

routes.py: Almacena las rutas relevantes a la API REST. Dentro de estas se almacena la lógica completa del manejo entidades generales, el manejo de solicitudes en formato JSON y la devolución de respuestas en el mismo formato después de interactuar con la base de datos. A continuación, se describe el funcionamiento de las rutas.

- **/usuarios [GET, POST]:** Maneja las entradas de la entidad ‘Usuarios’. Al utilizar el método GET, devuelve la lista de todos los usuarios registrados en el sistema, omitiendo mostrar las columnas “PasswordHash” y “Sal” para mantener la seguridad de los usuarios (además de ser objetos varbinary(64) y uniqueidentifier, respectivamente, los que no se pueden colocar directamente en JSON). Al utilizar el método POST, se espera recibir como entrada una solicitud JSON con las columnas “nombre”, “username”, “email” y “password”, utilizados para registrar

a un nuevo usuario en la entidad. Si se omite una de estas columnas, incluyen una entrada de cadena vacía o la longitud de una cadena excede a lo permitido en la base de datos, se devuelve un mensaje de error informando al usuario. De lo contrario, se confirma el registro y se muestra la ID del usuario recién creado.

- **/usuarios/<int:id_usuario> [GET, PUT, DELETE]:** Maneja una entrada individual de la entidad ‘Usuarios’. Al utilizar el método GET, busca un usuario con la ID proporcionada en la ruta dentro de la entidad mencionada. Si no se encuentra se devuelve un mensaje de error informando al usuario. De lo contrario, se devuelve la información del usuario encontrado, omitiendo las columnas “PasswordHash” y “Sal” por las razones mencionadas en la ruta anterior. Al utilizar el método PUT, se revisa la existencia del usuario buscado con la id proporcionada. De encontrarlo, se verifica que se haya proporcionado un JSON de entrada con las columnas “nombre”, “username” y “email”, campos que el usuario podrá actualizar. Se omite el campo “password”, pues requeriría un nuevo cifrado de la contraseña, tarea que se manejaría de forma separada (no implementada en este programa, pero incluida en la base de datos). Si no se encuentran los campos apropiados o se encuentran fallos de lógica, se devuelve un error informando al usuario de la causa. De lo contrario, se confirma la actualización de los valores del usuario. Al utilizar el método DELETE, se busca al usuario con la id proporcionada, y, de encontrarla, se elimina, marcando su columna “Activo” como false y bloqueando su modificación posterior. Esto asegura la integridad referencial al trabajar con entidades conectadas a la entidad usuario (Ordenes). Se le informa al usuario de la eliminación de la entrada.

- **/productos [GET, POST]:** Maneja las entradas de la entidad ‘Producto’. Al utilizar el método GET, devuelve la lista de todos los productos registrados en el sistema. Al utilizar el método POST, se espera recibir como entrada una solicitud JSON con las columnas “nombre_producto”, “precio” y “disponibilidad”, utilizados para registrar un nuevo producto en la entidad. Si se omite una de estas columnas, incluyen una entrada de cadena vacía, la longitud de una cadena excede a lo permitido en la base de datos o los números presentan inconsistencias lógicas (precio negativo o igual a 0), se devuelve un mensaje de error informando al usuario. De lo contrario, se confirma el registro y se muestra la ID del producto registrado.
- **/productos/<int:id_producto> [GET, PUT, DELETE]:** Maneja una entrada individual de la entidad ‘Producto’. Al utilizar el método GET, busca un producto con la ID proporcionada en la ruta dentro de la entidad mencionada. Si no se encuentra se devuelve un mensaje de error informando al usuario. De lo contrario, se devuelve la información del producto encontrado. Al utilizar el método PUT, se revisa la existencia del producto buscado con la id proporcionada. De encontrarlo, se verifica que se haya proporcionado un JSON de entrada con las columnas “nombre_producto”, “precio” y “disponible”, campos que el usuario podrá actualizar. Si no se encuentran los campos apropiados o se encuentran fallos de lógica, se devuelve un error informando al usuario de la causa. De lo contrario, se confirma la actualización de los valores del producto. Al utilizar el método DELETE, se busca el producto con la id proporcionada, y, de encontrarlo,

se elimina, marcando su columna “Activo” como false y bloqueando su modificación posterior.

- **/ordenes [GET, POST]:** Maneja las entradas de la entidad ‘Orden’. Al utilizar el método GET, devuelve la lista de todas las órdenes registradas en el sistema. Al utilizar el método POST, se espera recibir como entrada una solicitud JSON con las columnas “id_usuario”, “id_producto”, “cantidad” y “fecha_hora_recoleccion”, utilizados para registrar una nueva orden en la entidad. Si se omite una de estas columnas, los números presentan inconsistencias lógicas (ids y cantidad negativa o igual a 0) o la fecha presenta un formato no consistente al formato datetime2(7) de Microsoft SQL Server, se devuelve un mensaje de error informando al usuario. De lo contrario, se confirma el registro y se muestra la ID de la orden registrada. Todas las nuevas órdenes empezaron con estado “En carrito”, modificando este por medio de rutas que se describirán posteriormente.
- **/ordenes/<int:id_orden> [GET, PUT, DELETE]:** Maneja una entrada individual de la entidad ‘Orden. Al utilizar el método GET, busca una orden con la ID proporcionada en la ruta dentro de la entidad mencionada. Si no se encuentra se devuelve un mensaje de error informando al usuario. De lo contrario, se devuelve la información de la orden encontrada. Al utilizar el método PUT, se revisa la existencia de la orden buscada con la id proporcionada. De encontrarla, se verifica que se haya proporcionado un JSON de entrada con las columnas “id_usuario”, “id_producto”, “cantidad” y “fecha_hora_recoleccion”, campos que el usuario podrá actualizar. Si no se encuentran los campos apropiados o se encuentran fallos de lógica, se devuelve un error informando al usuario de la

causa. De lo contrario, se confirma la actualización de los valores de la orden. Al utilizar el método DELETE, se busca la orden con la id proporcionada, y, de encontrarla, se elimina, marcando su columna “Activo” como false y bloqueando su modificación posterior.

- **/ordenes/confirmar/<int:id_orden> [PUT]:** Confirma una orden existente, cambiando su estado a “Confirmado”. Primero, se verifica la existencia de la orden buscada con la id de la ruta. Si no se encuentra, se devuelve un error informando al usuario. Si se encuentra al usuario, se busca un pago relacionado a la orden (es necesario confirmar al menos la existencia de un método de pago antes de confirmar la orden). Si no existe, se devuelve un error al usuario. De lo contrario, se intenta confirmar la orden. Se actualiza el estado de la orden o se devuelve un error si esta no se completa exitosamente.
- **/ordenes/completar /<int:id_orden> [PUT]:** Completa una orden confirmada, cambiando su estado a “Completada”. Primero, verifica la existencia de una orden, devolviendo un error si no se descubre una con la ruta especificada. Después, se verifica si la orden es válida para completarse (cuenta con el estado “Confirmada”). Si no es válida, se devuelve un error. De lo contrario, se regresa un mensaje de éxito, actualizando el estado de la orden a “Completada” y marcándola como “Activo”.
- **/pagos [GET, POST]:** Maneja las entradas de la entidad ‘Pago’. Al utilizar el método GET, devuelve la lista de todos los pagos registrados en el sistema. Al utilizar el método POST, se espera recibir como entrada una solicitud JSON con las columnas “id_orden”, “id_metodo_pago” y “cantidad_pagada”, utilizados para

registrar un nuevo pago en la entidad. La cantidad por pagar se determina de la orden introducida, multiplicando la cantidad de la orden por el precio del producto. Si se omite una de estas columnas, los números presentan inconsistencias lógicas (ids y cantidad pagada negativa o igual a 0) o la cantidad pagada es menor que la cantidad por pagar, se devuelve un mensaje de error informando al usuario. De lo contrario, se confirma el registro y se muestra la ID del pago registrado. El pago requiere una orden existente para registrarse correctamente.

- **/pagos /<int:id_pago> [GET, PUT, DELETE]:** Maneja una entrada individual de la entidad 'Pago'. Al utilizar el método GET, busca un pago con la ID proporcionada en la ruta dentro de la entidad mencionada. Si no se encuentra se devuelve un mensaje de error informando al usuario. De lo contrario, se devuelve la información del pago encontrado. Al utilizar el método PUT, se revisa la existencia del pago buscado con la id proporcionada. De encontrarlo, se verifica que se haya proporcionado un JSON de entrada con las columnas "id_orden", "id_metodo_pago" y "cantidad_pagada", campos que el usuario podrá actualizar. Si no se encuentran los campos apropiados o se encuentran fallos de lógica (incluyendo las consideraciones explicadas en la ruta anterior), se devuelve un error informando al usuario de la causa. De lo contrario, se confirma la actualización de los valores del pago. Al utilizar el método DELETE, se busca el producto con la id proporcionada, y, de encontrarlo, se elimina de la base de datos.

Pruebas

Para probar el sistema, veremos el flujo de trabajo para un cliente y un administrador en la aplicación web, además de observar el funcionamiento de la API REST y todas las operaciones. Para reducir la cantidad de capturas de pantalla repetitivas, se demostrará únicamente un caso de los diferentes errores que pueden ocurrir en los programas, dando prioridad a errores exclusivos a cada operación.

Aplicación web

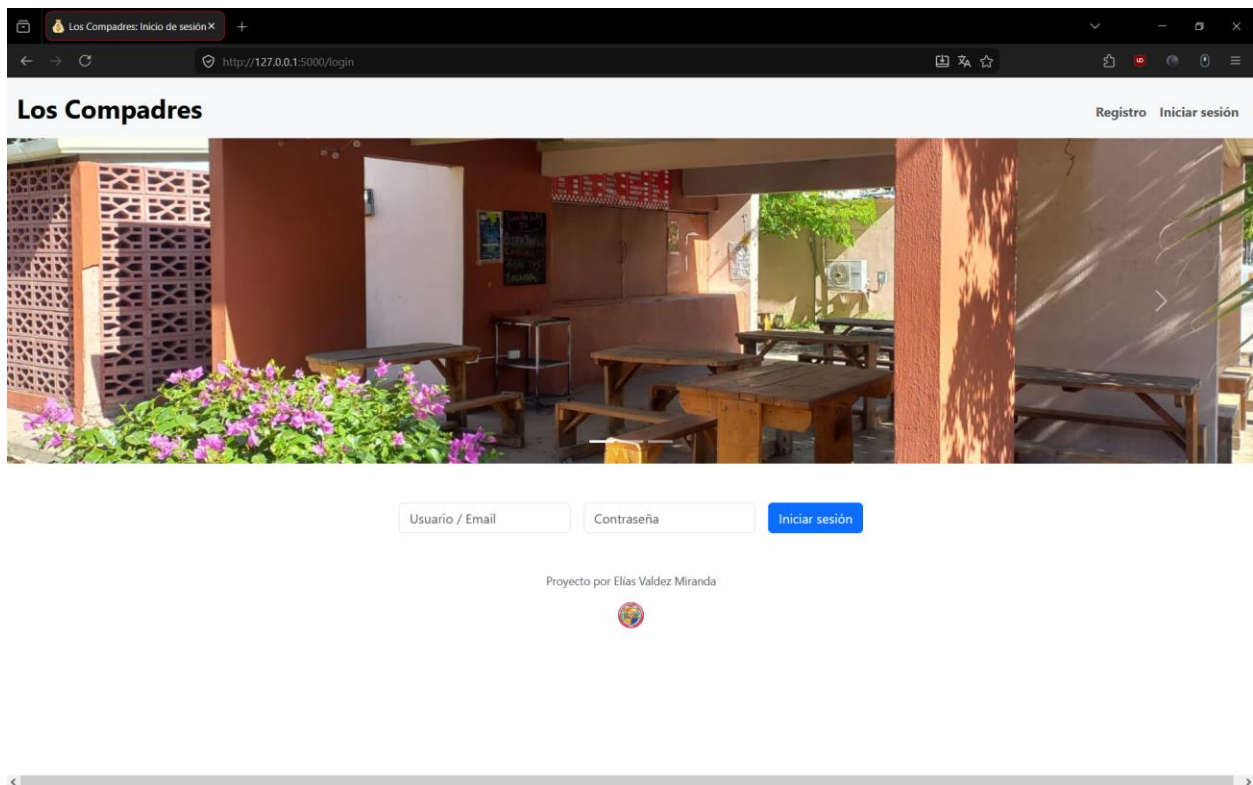


Imagen 01: Pantalla de inicio

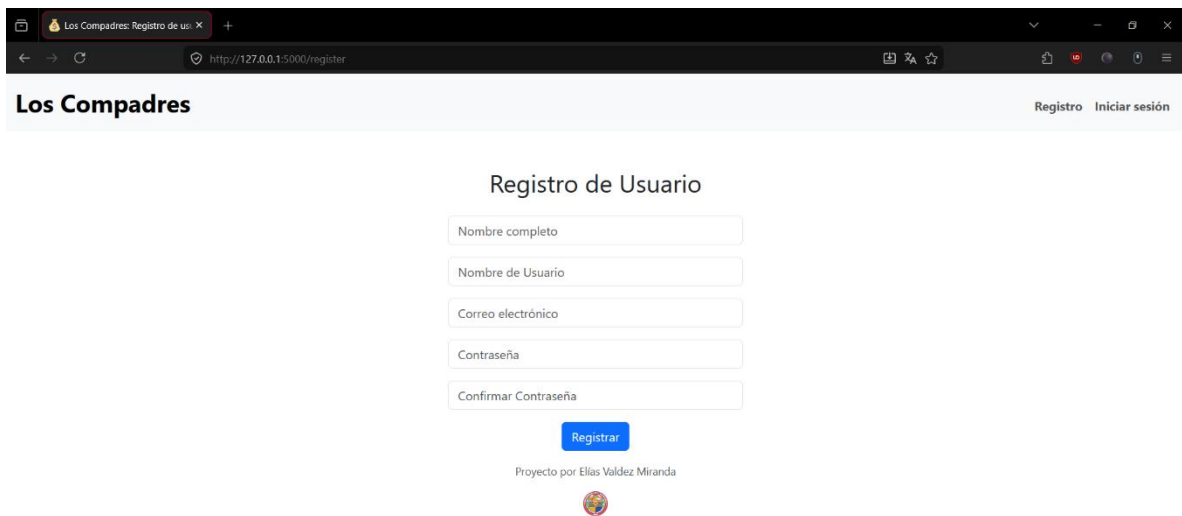


Imagen 02: Pantalla de registro de usuario

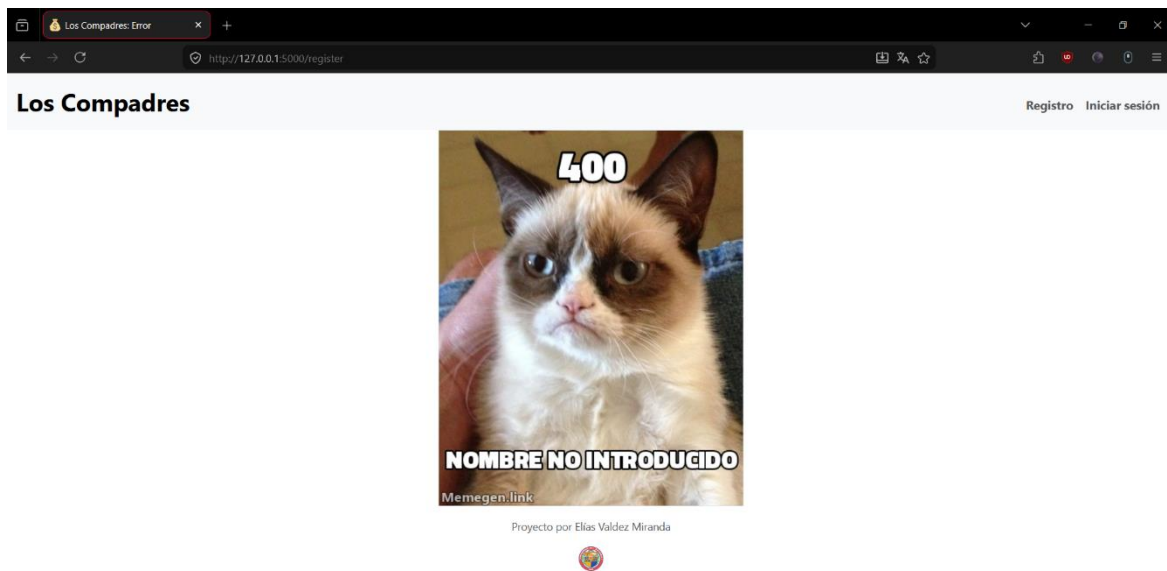


Imagen 03: Error en registro de usuario (Nombre legal no introducido al enviar formulario)

Los Compadres Registro Iniciar sesión

Registro de Usuario

Elias Valdez Miranda

EliasVM

eliasvaldezmiranda14@gmail.com

••••••••

••••••••

Registrar

Proyecto por Elias Valdez Miranda

Imagen 04: Formulario completo

Los Compadres Agregar orden Historial Cerrar sesión

¡Bienvenido EliasVM!

ID	Producto	Precio (unidad)	Cantidad	Precio Total	Estado	Hora de recolección	Acciones
No hay órdenes activas en este momento.							

Proyecto por Elias Valdez Miranda

Imagen 05: Pantalla principal del sistema (ventana de órdenes activas)

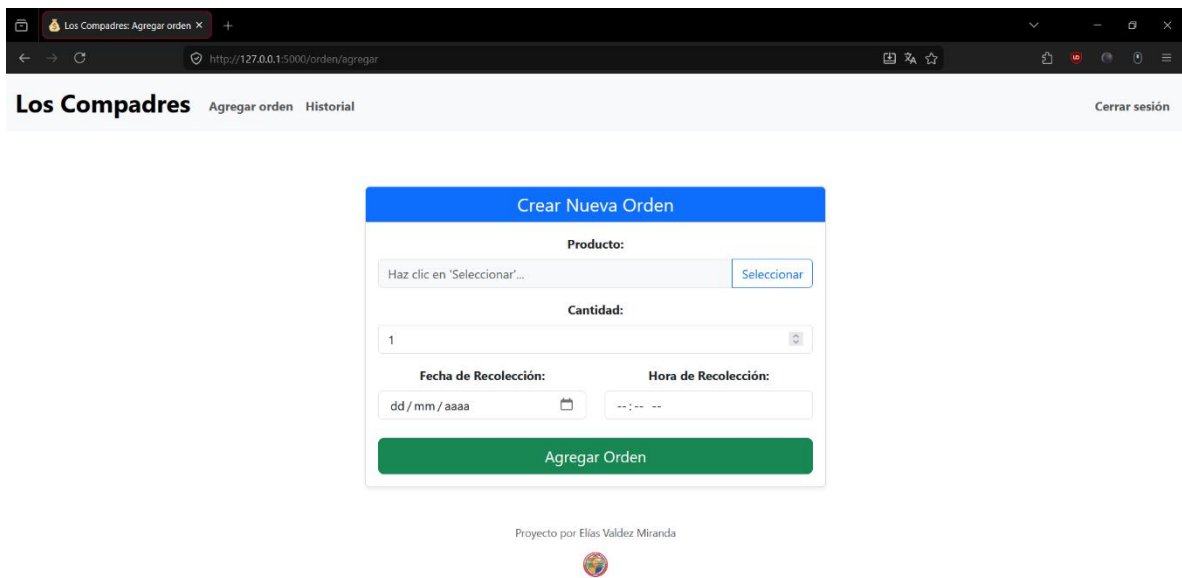


Imagen 06: Pantalla de creación de nueva orden

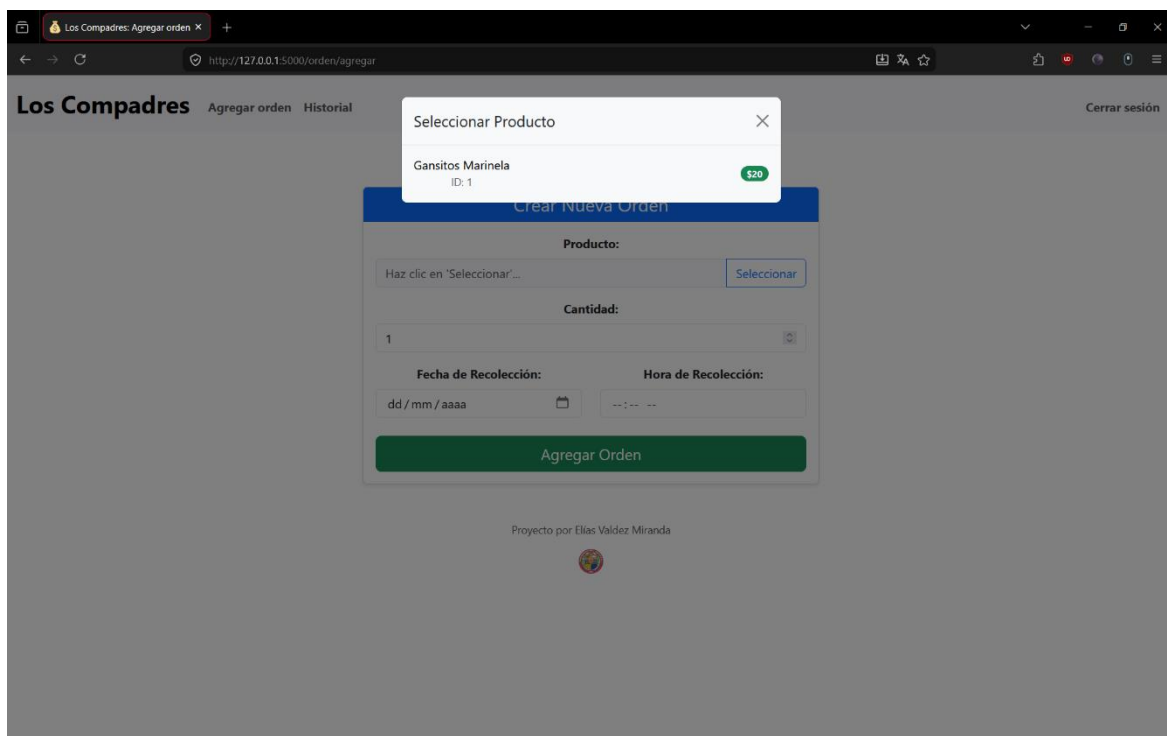


Imagen 07: Listado de selección de productos

Los Compadres Agregar orden Historial Cerrar sesión

Crear Nueva Orden

Producto:
Gansitos Marinela (\$20) Seleccionar

Cantidad:
2

Fecha de Recolección: 13/05/2026 **Hora de Recolección:** 07:00 a.m.

Agregar Orden

Proyecto por Elías Valdez Miranda

Imagen 08: Formulario de orden completo

Los Compadres Agregar orden Historial Cerrar sesión

¡Bienvenido EliasVM!

ID	Producto	Precio (unidad)	Cantidad	Precio Total	Estado	Hora de recolección	Acciones
2	Gansitos Marinela	\$20.00	2	\$40.00	En carrito	2026-05-13 07:00:00	Actualizar orden

Proyecto por Elías Valdez Miranda

Imagen 09: Resultado de la inserción de la orden

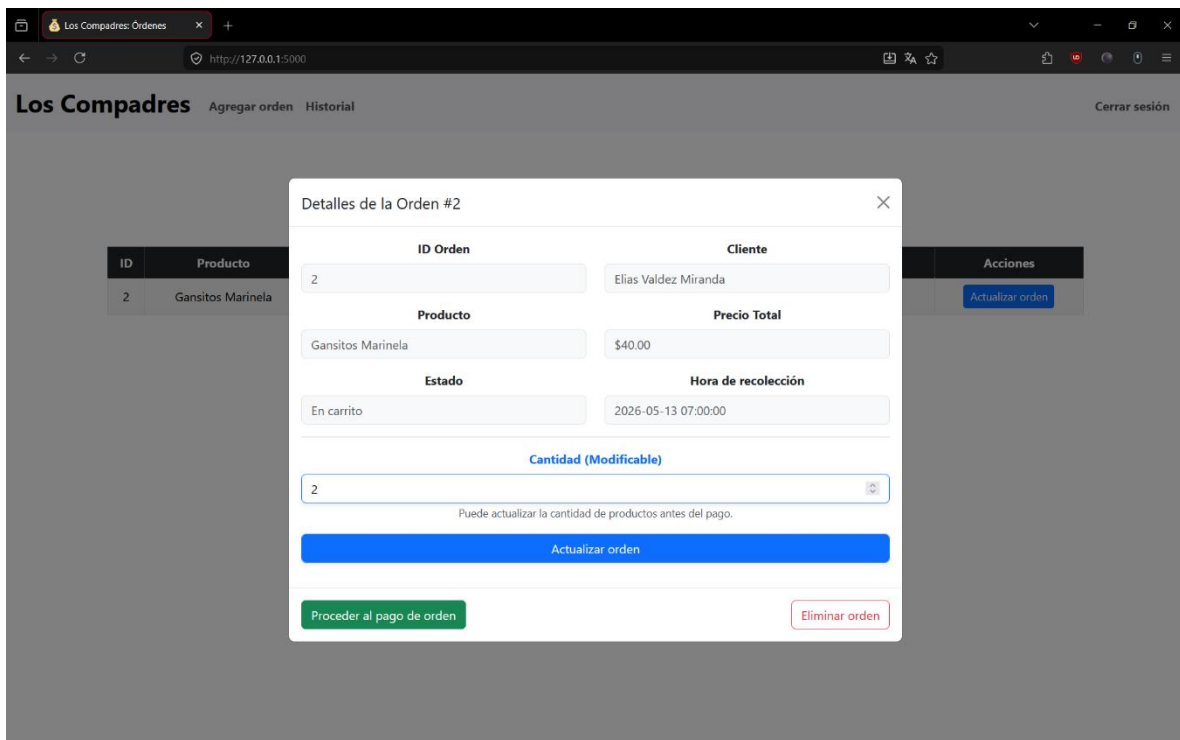


Imagen 10: Pantalla de modificación de orden

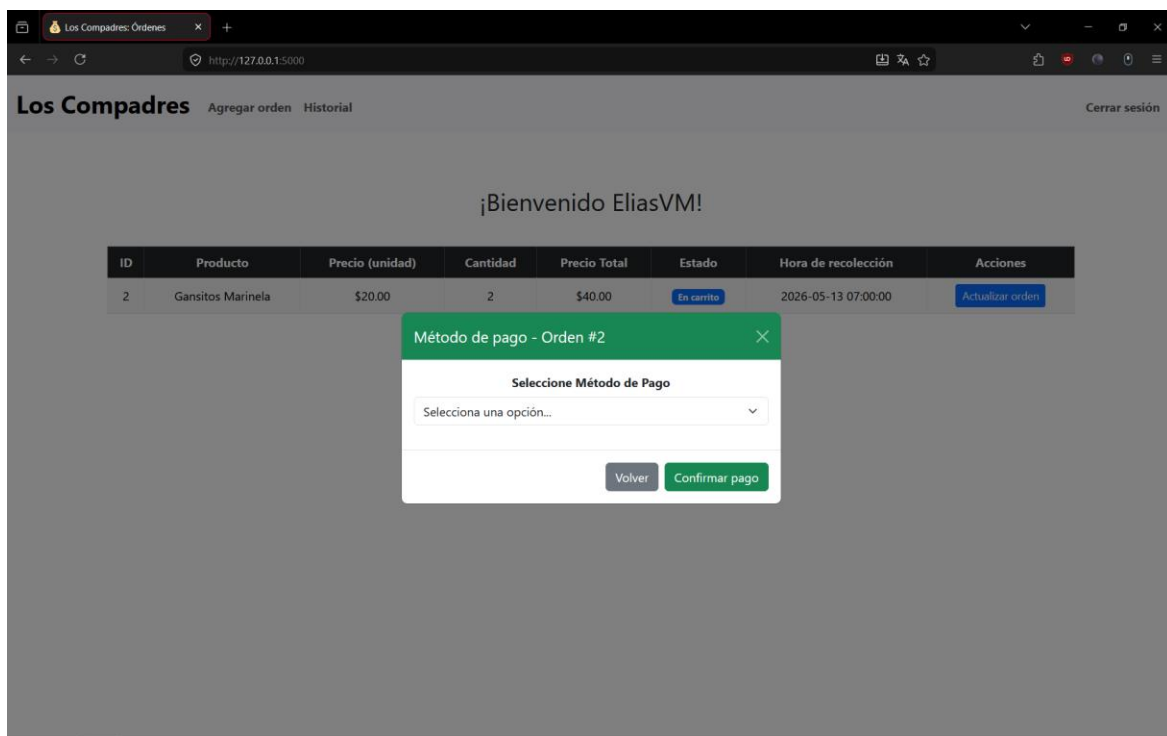


Imagen 11: Pantalla de pago de orden

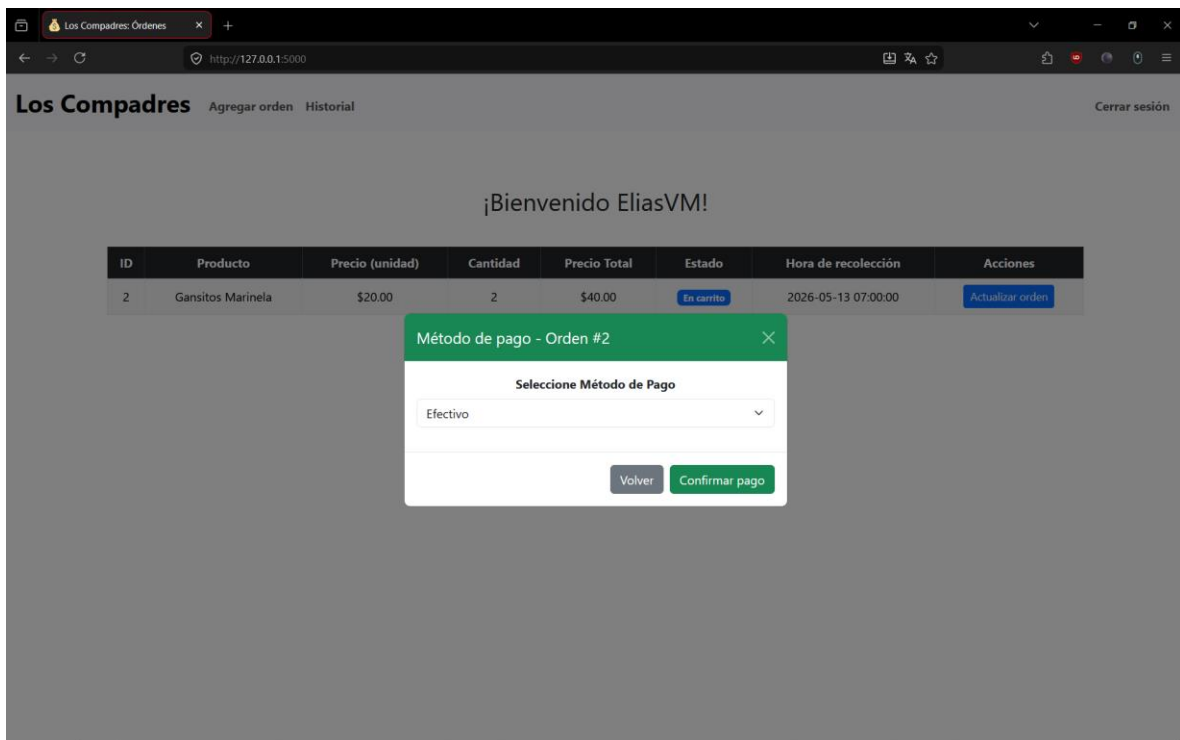


Imagen 12: Selección de pago de orden por efectivo

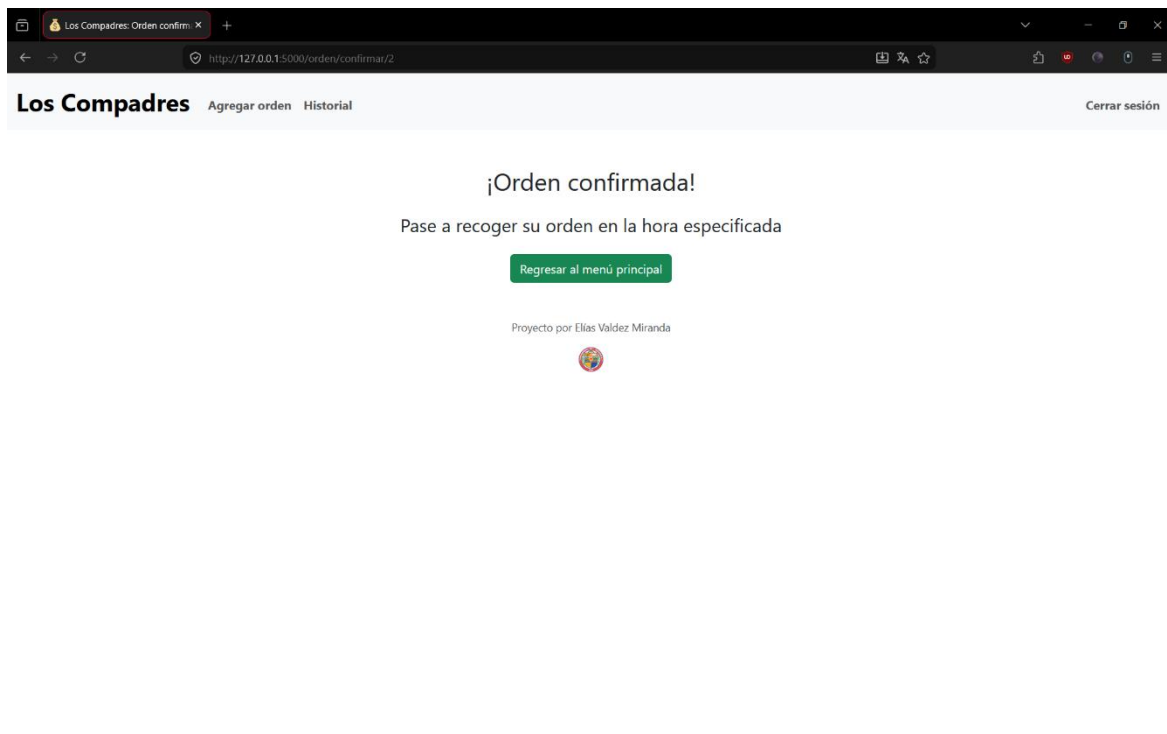


Imagen 13: Confirmación de orden



Imagen 14: Pantalla con orden confirmada

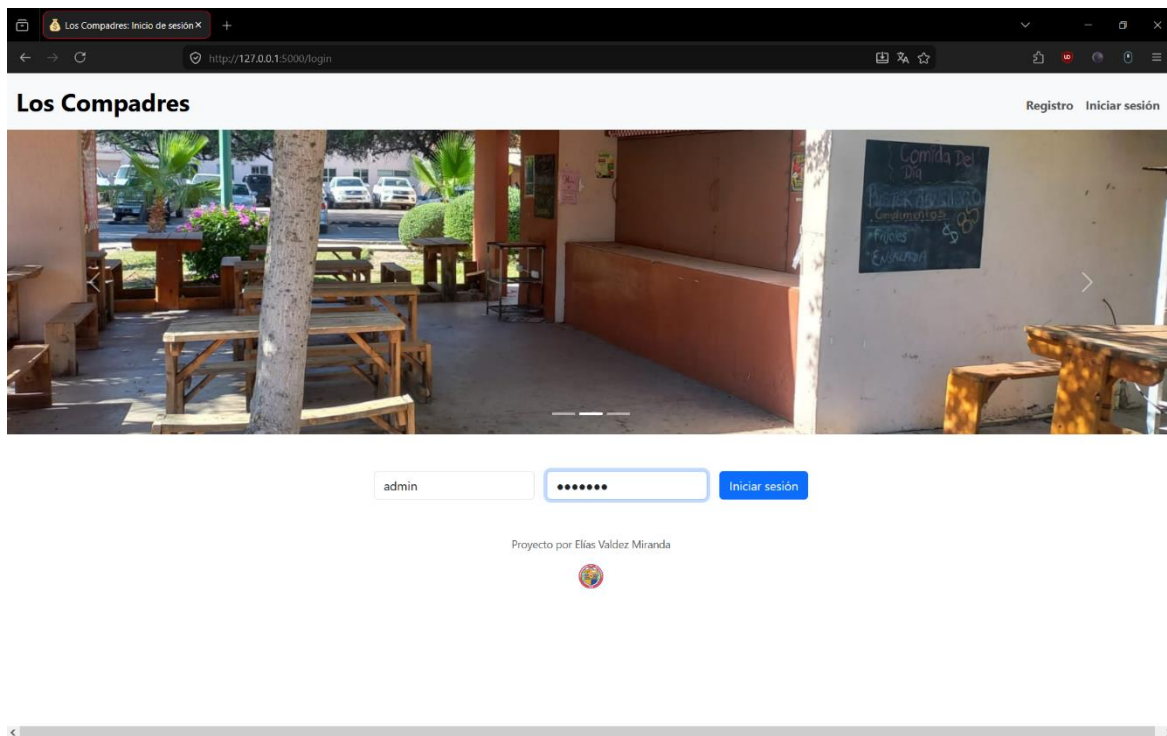


Imagen 15: Pantalla de inicio con credenciales de administrador incorrectas

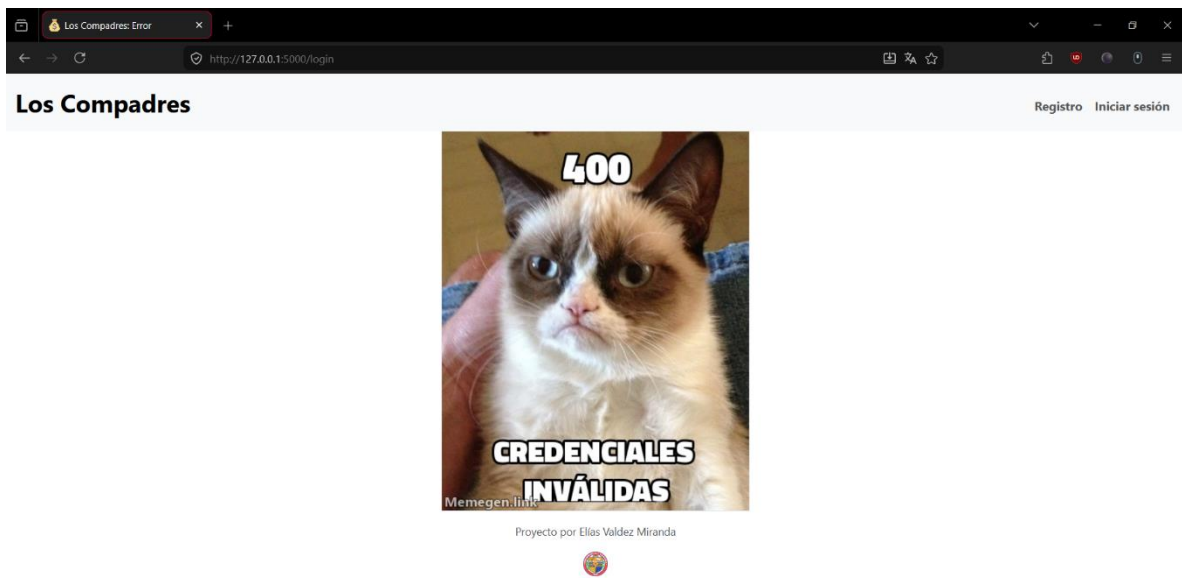


Imagen 16: Error de credenciales inválidas

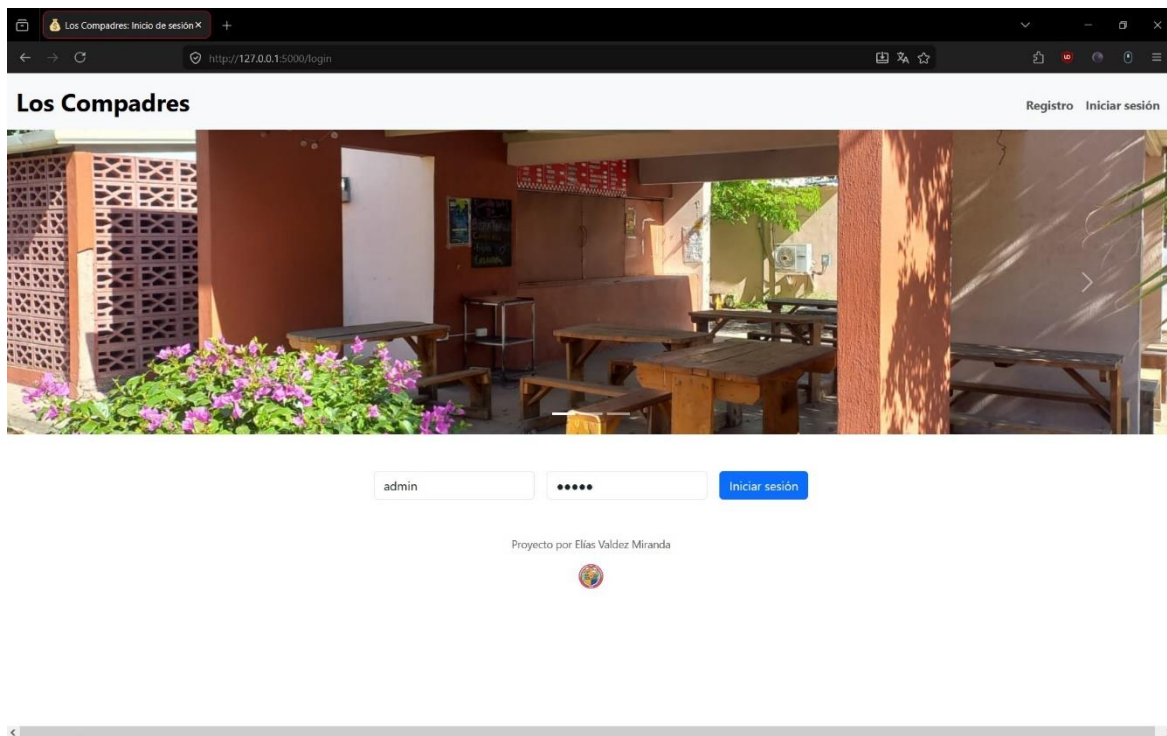


Imagen 17: Pantalla de inicio con credenciales de administrador correctas

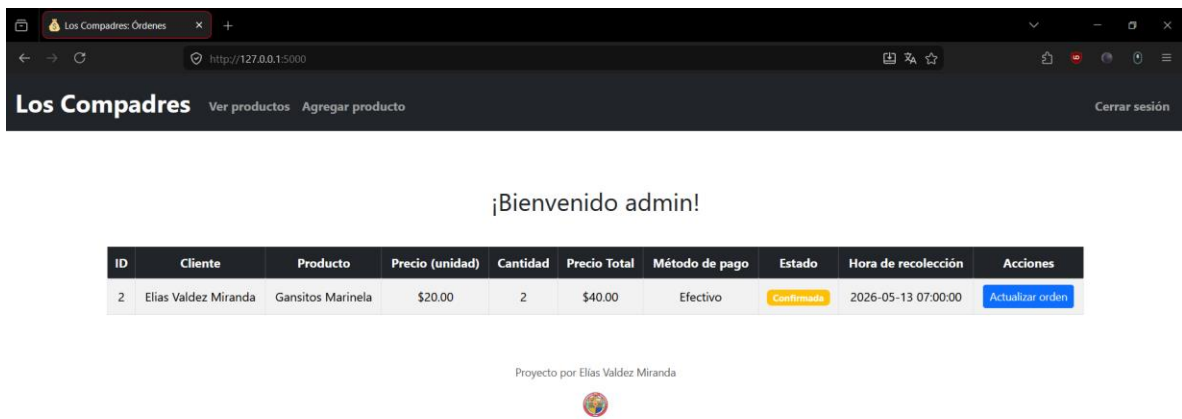


Imagen 18: Pantalla principal del sistema (ventana de órdenes confirmadas)

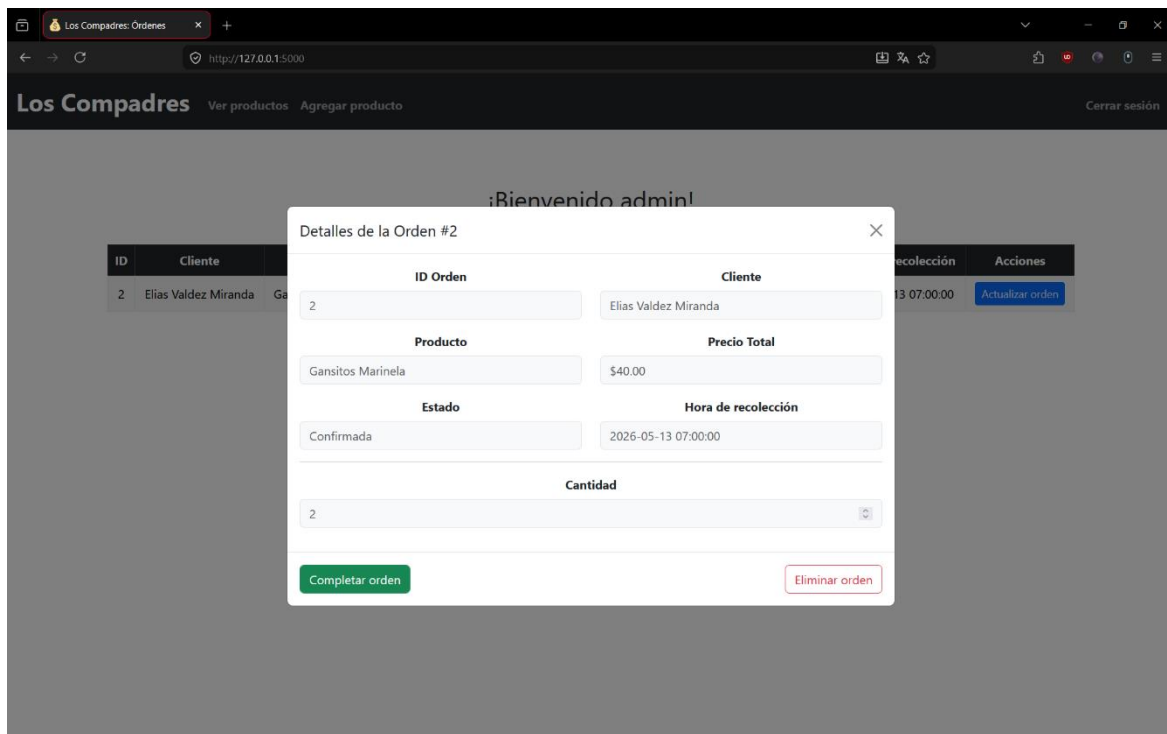


Imagen 19: Pantalla de actualización de orden confirmada

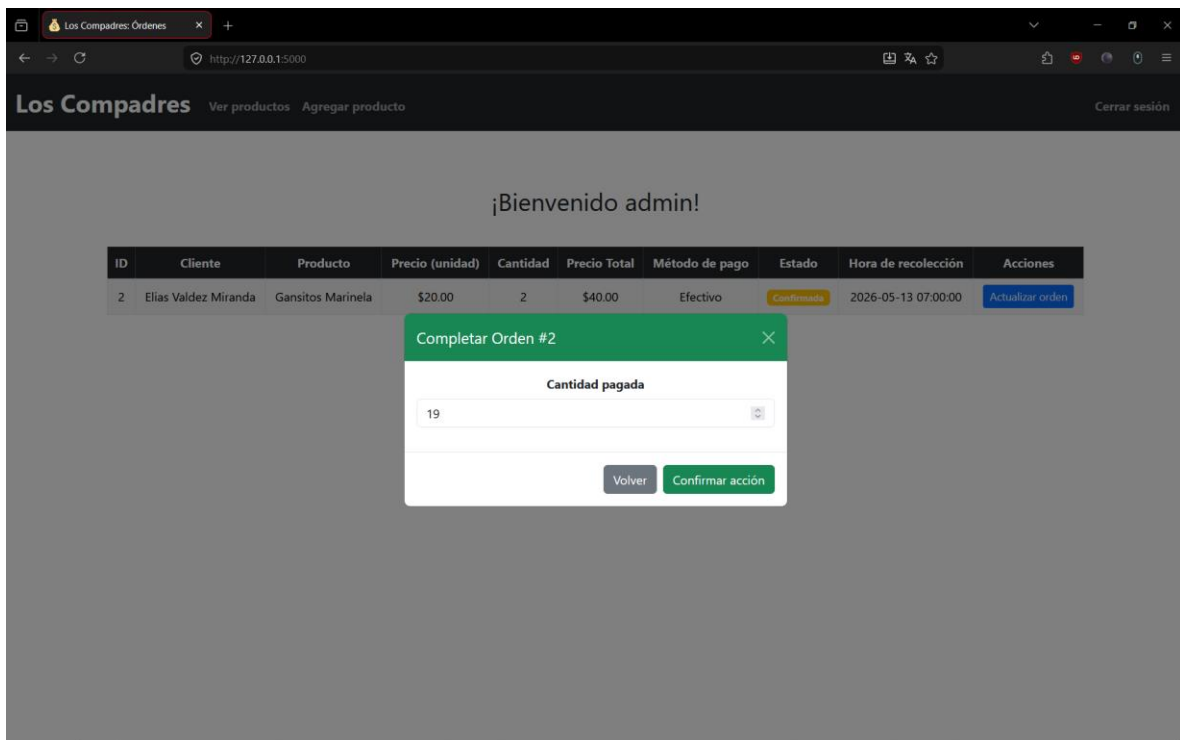


Imagen 20: Pantalla de confirmación de pago (insuficiente dinero)

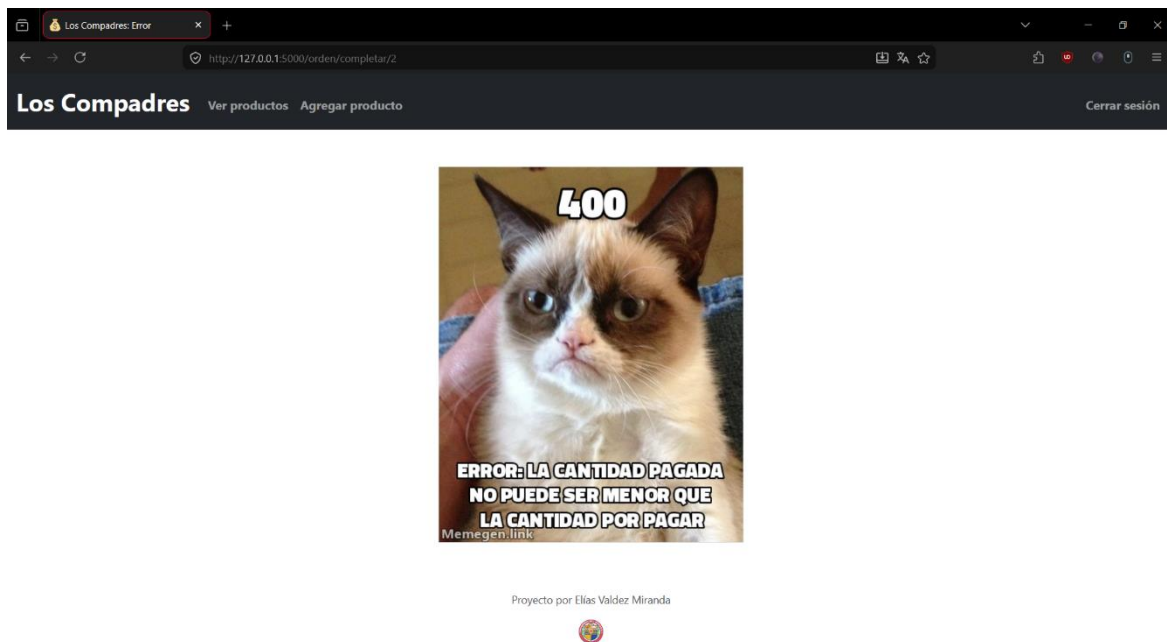


Imagen 21: Error en completar el pago (la cantidad de dinero pagada es insuficiente)

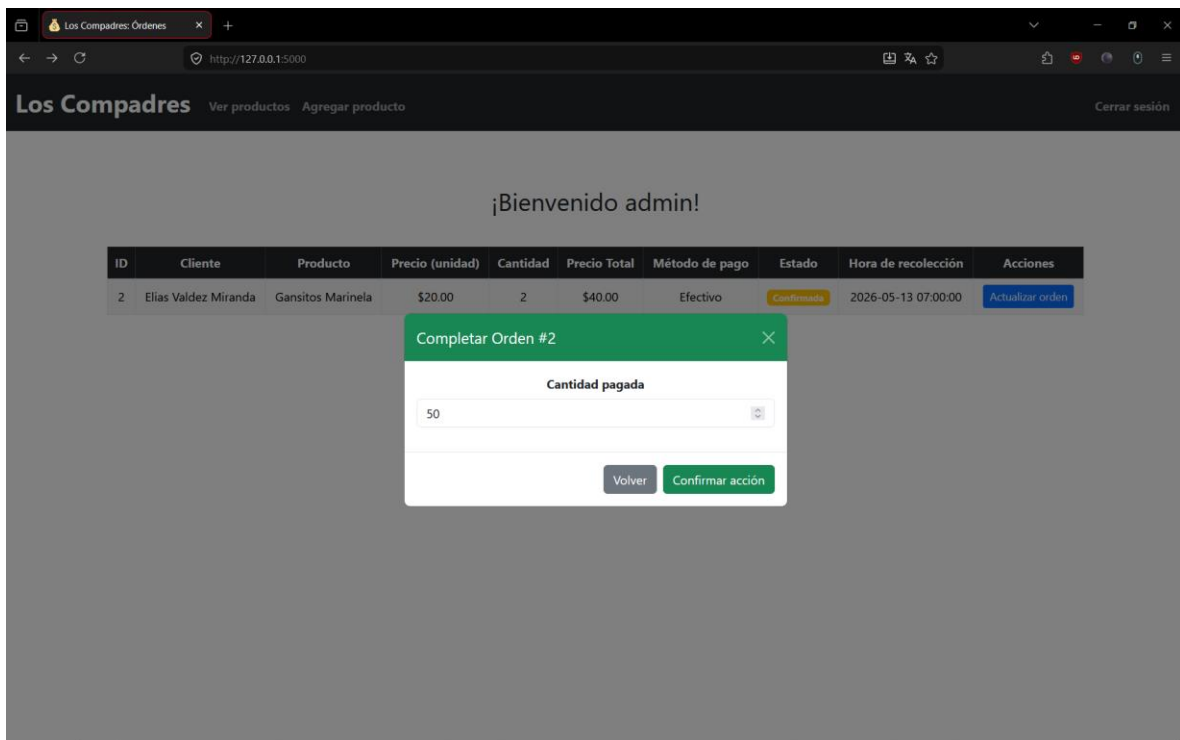


Imagen 22: Pantalla de confirmación de pago (suficiente dinero)

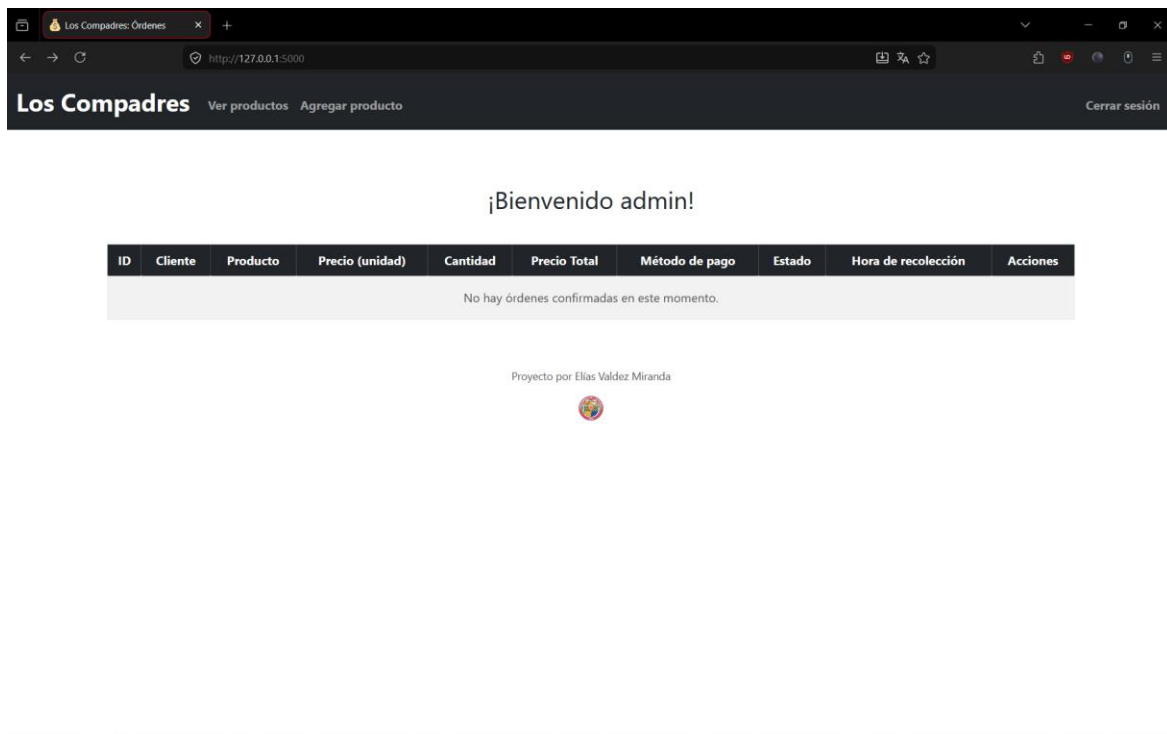


Imagen 23: Pago de orden completada (la orden pasa a estar “completa”)

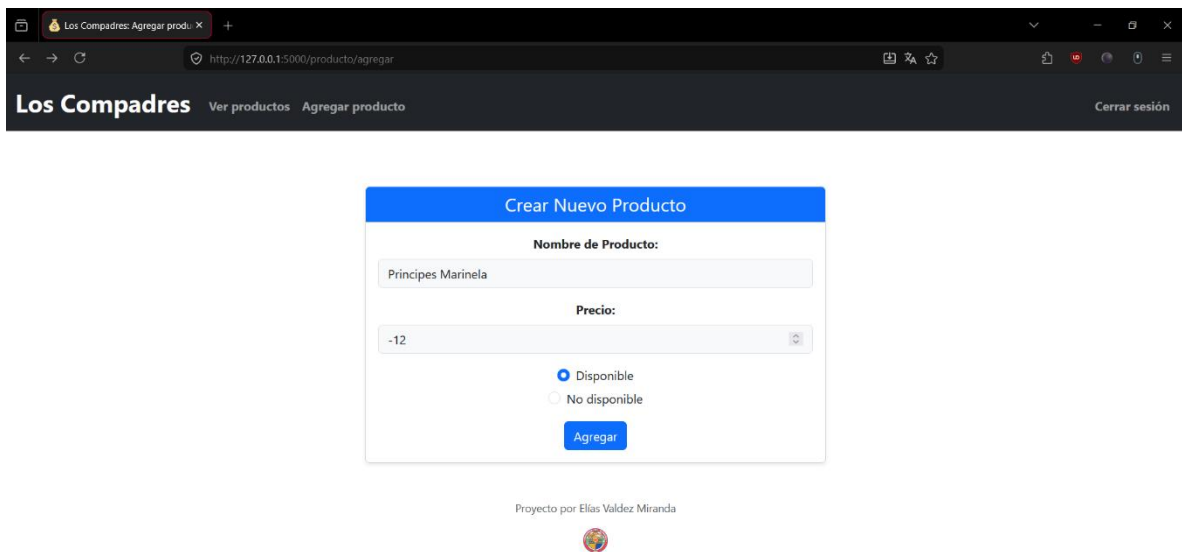


Imagen 24: Pantalla de registro de producto (precio incorrecto)

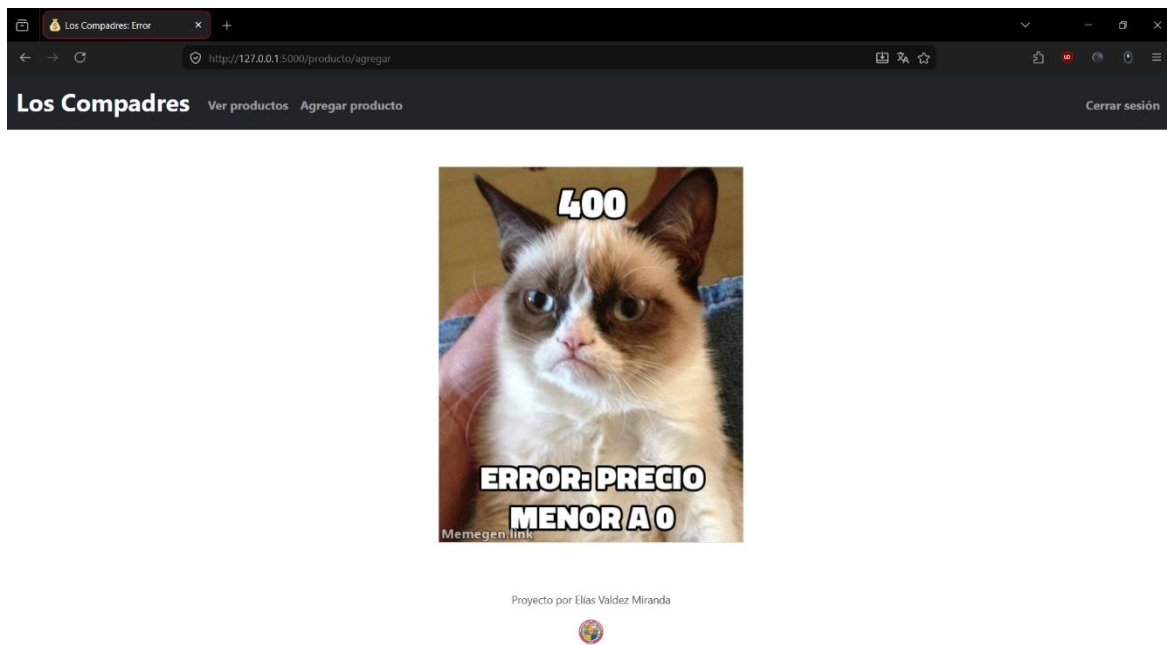


Imagen 25: Error en registro de producto (Precio menor a \$0)

Los Compadres Ver productos Agregar producto Cerrar sesión

Crear Nuevo Producto

Nombre de Producto:

Principes Marinela

Precio:

22

☒ Disponible
☐ No disponible

Agregar

Proyecto por Elías Valdez Miranda

Imagen 26: Pantalla de registro de producto (valores correctos)

Los Compadres Ver productos Agregar producto Cerrar sesión

Ver Productos

ID	Nombre del Producto	Precio	Disponibilidad	Acciones
1	Gansitos Marinela	\$20.00	Disponible	Editar
3	Principes Marinela	\$22.00	Disponible	Editar

Proyecto por Elías Valdez Miranda

Imagen 27: Pantalla de visualización de productos

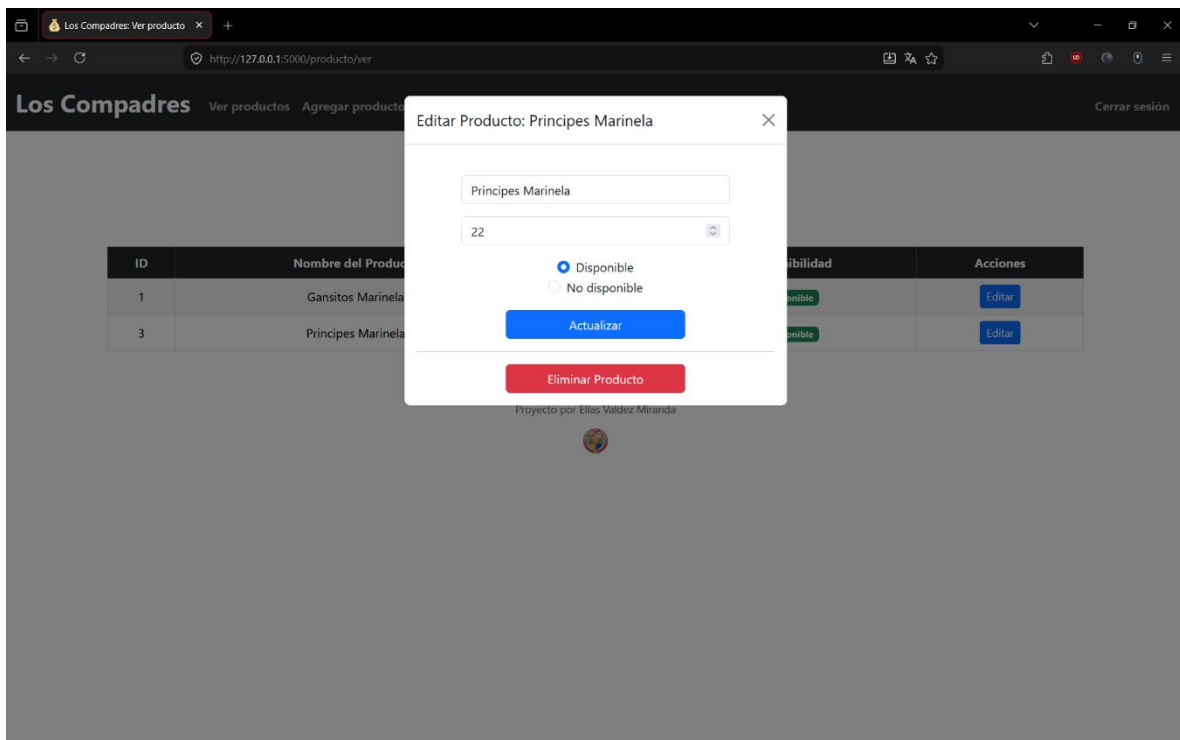


Imagen 28: Pantalla de edición de producto

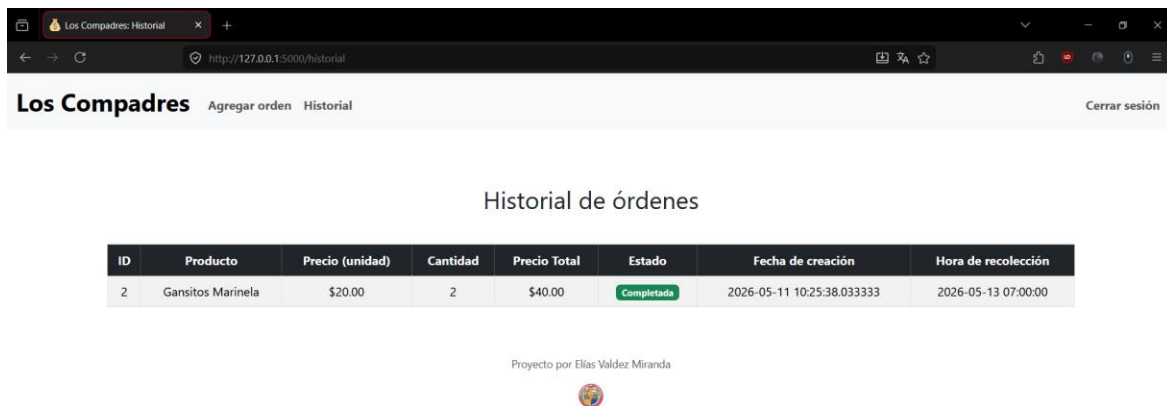


Imagen 29: Pantalla de visualización de historial (cliente)

API REST

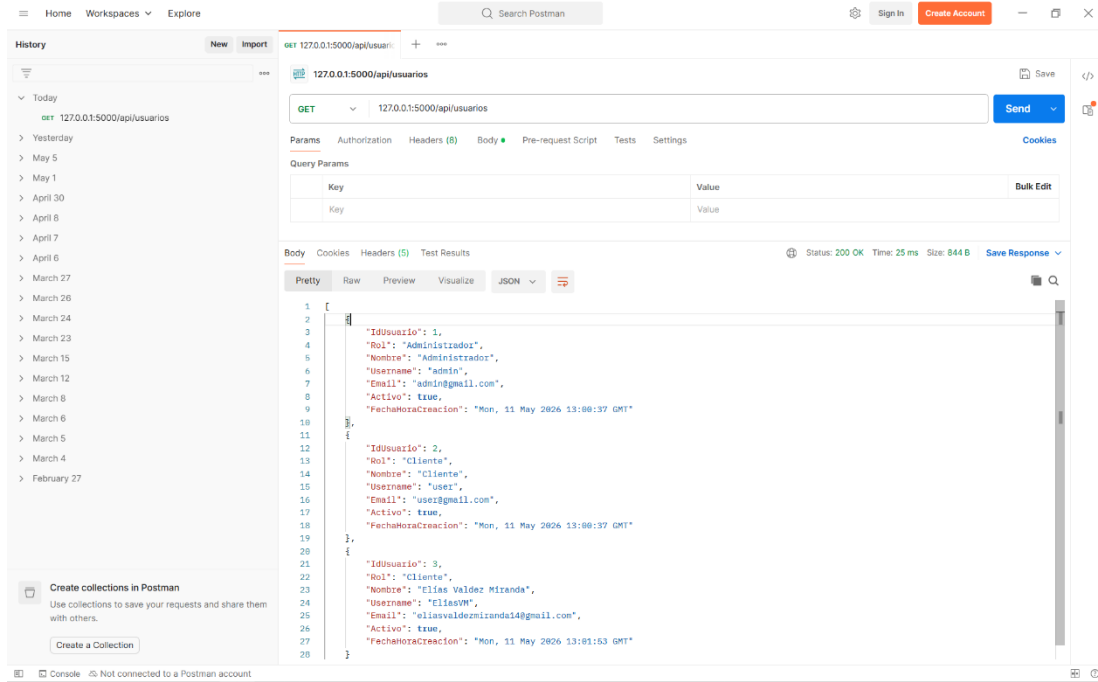


Imagen 30: Solicitud GET exitosa a la entidad Usuarios

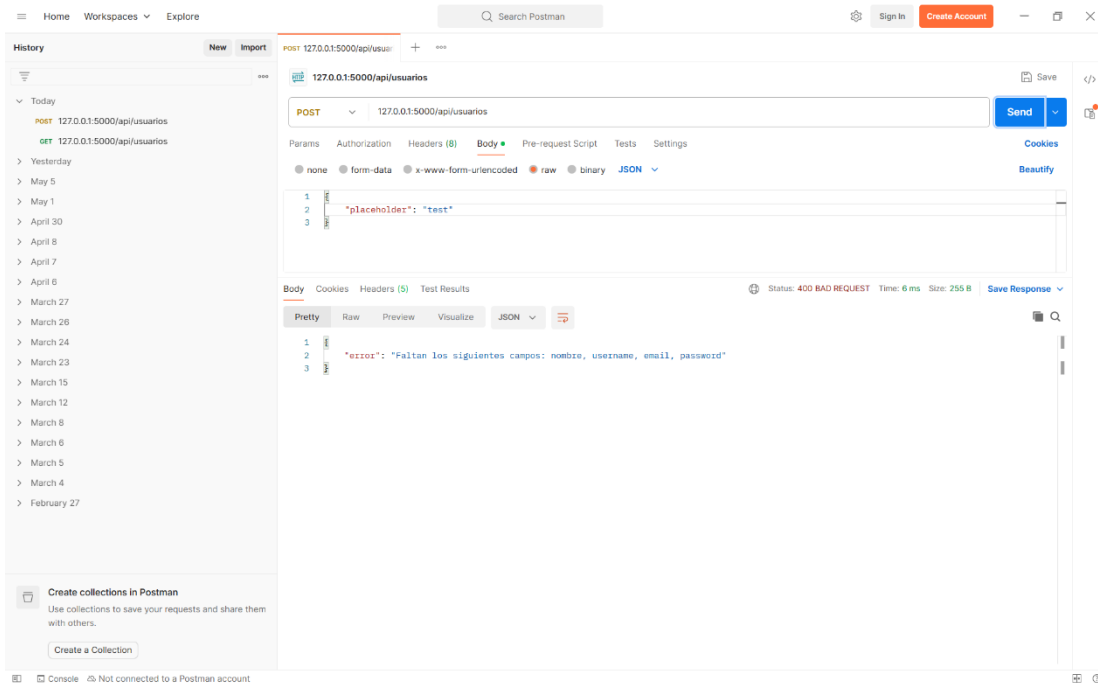


Imagen 31: Solicitud POST err\u00f3nea (campos ausentes)

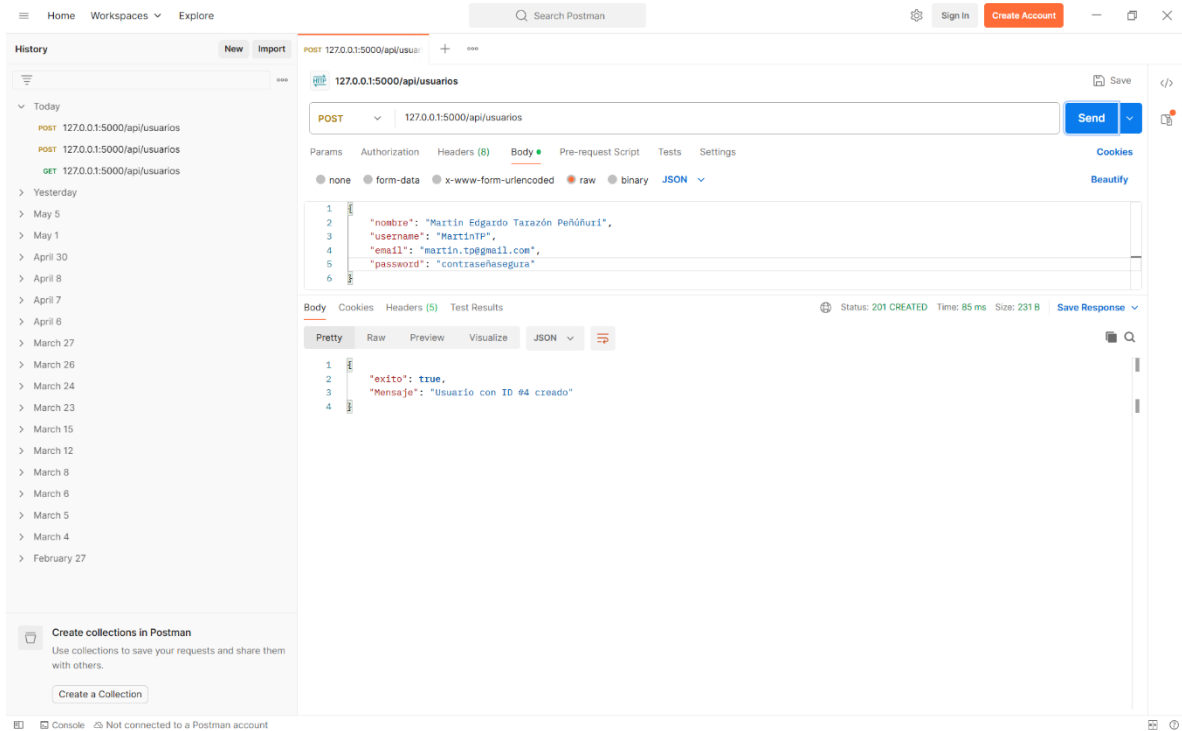


Imagen 32: Solicitud POST exitosa a la entidad Usuarios

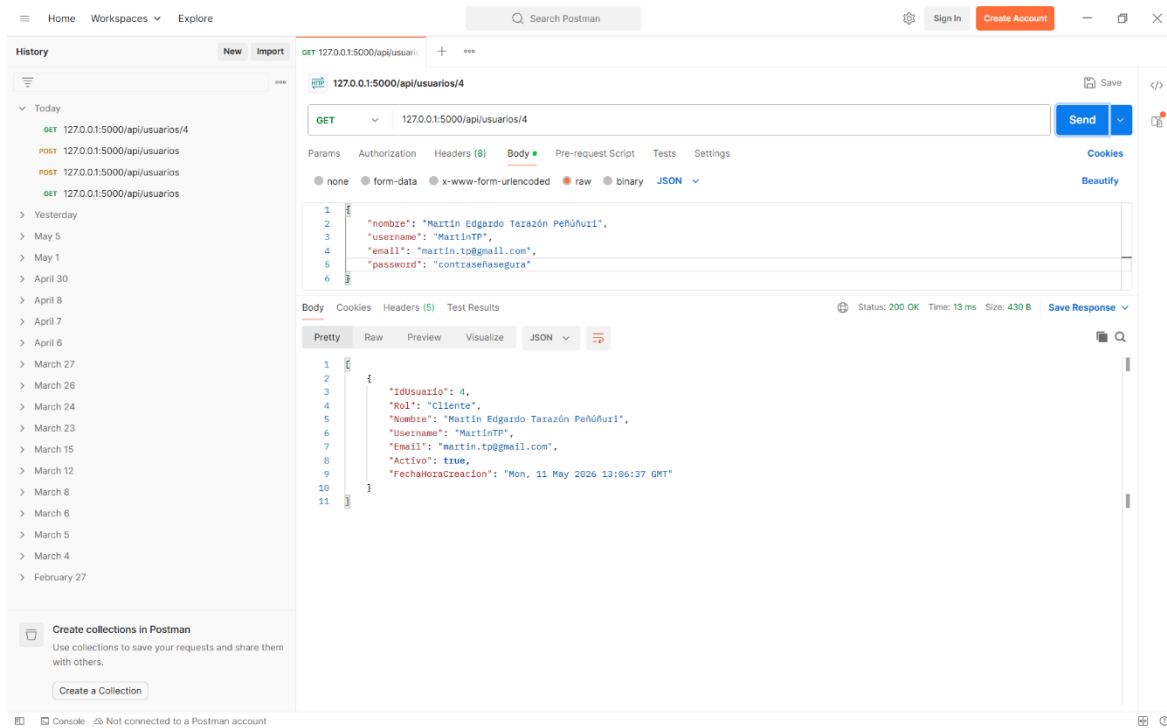


Imagen 33: Solicitud GET por ID exitosa a la entidad Usuarios

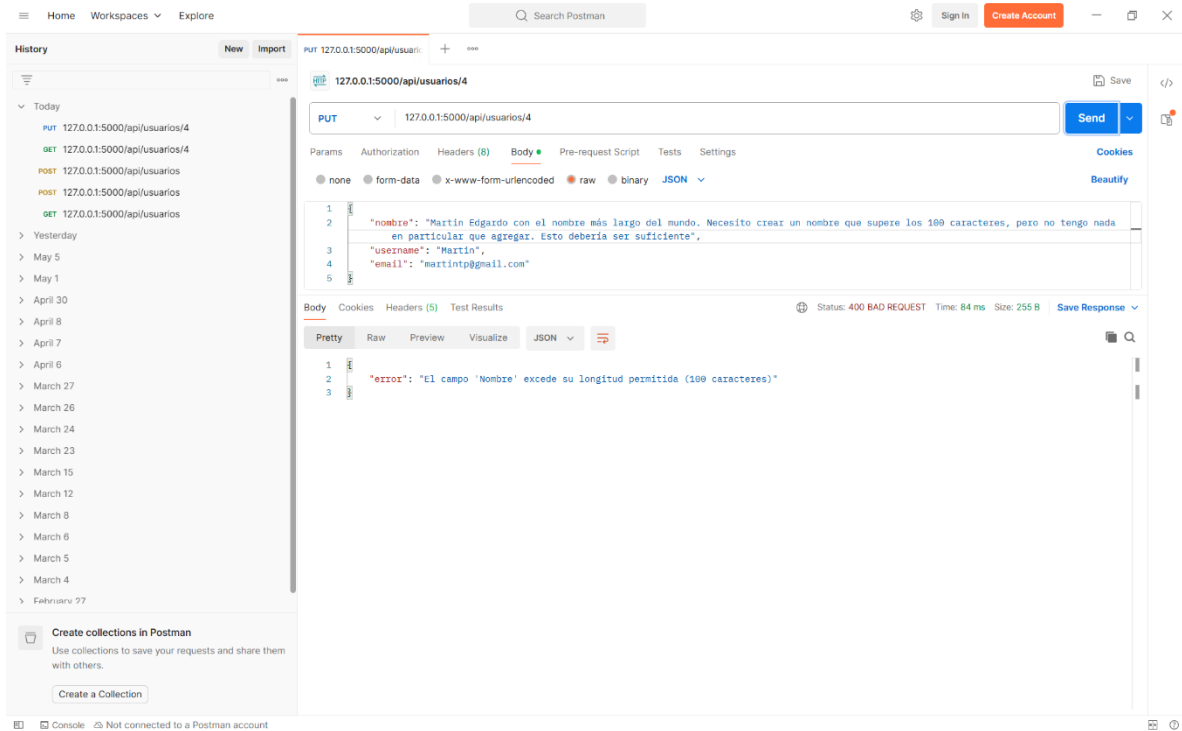


Imagen 34: Solicitud PUT err\u00f3nea (cadena mayor al l\u00edmite de caracteres)

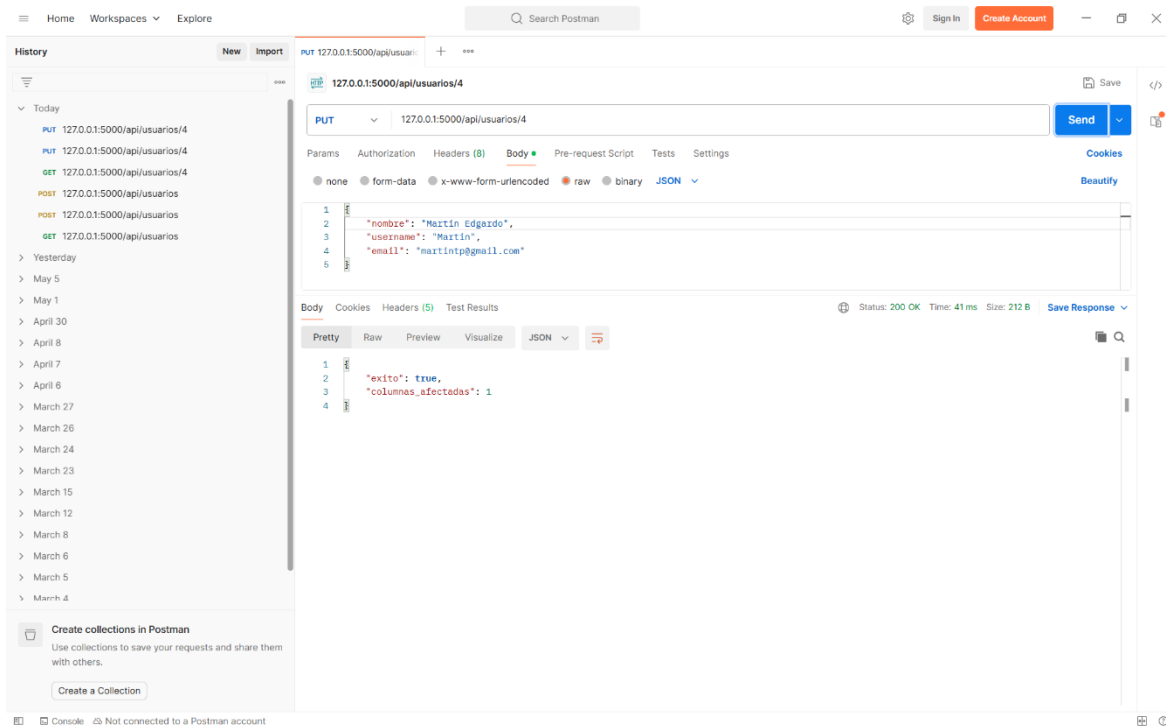


Imagen 35: Solicitud PUT por ID exitosa a la entidad Usuarios

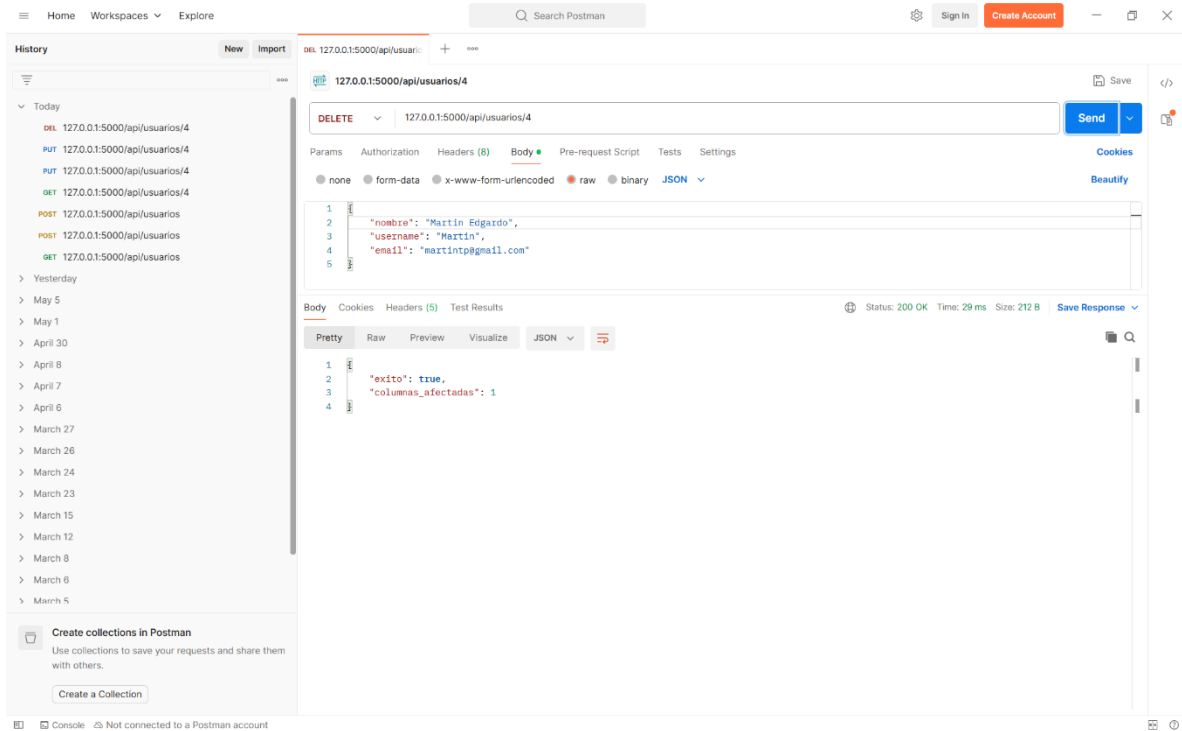


Imagen 36: Solicitud DELETE por ID exitosa a la entidad Usuarios

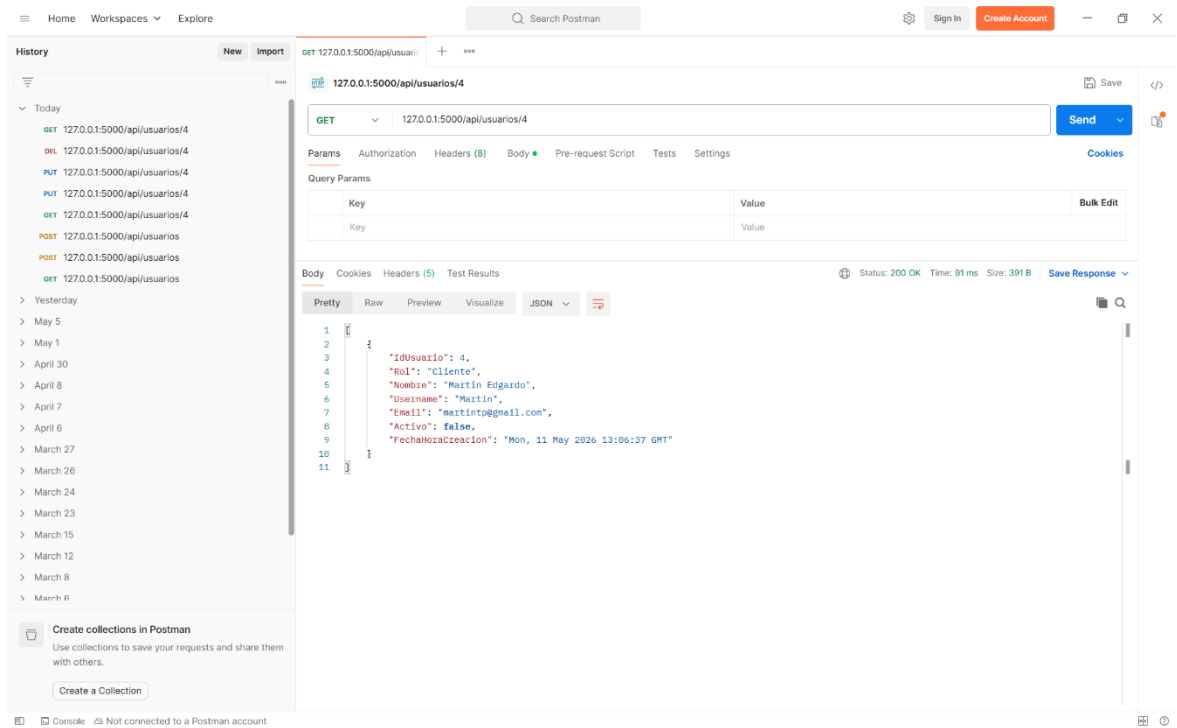


Imagen 37: Verificación de eliminación exitosa (Activo = false)

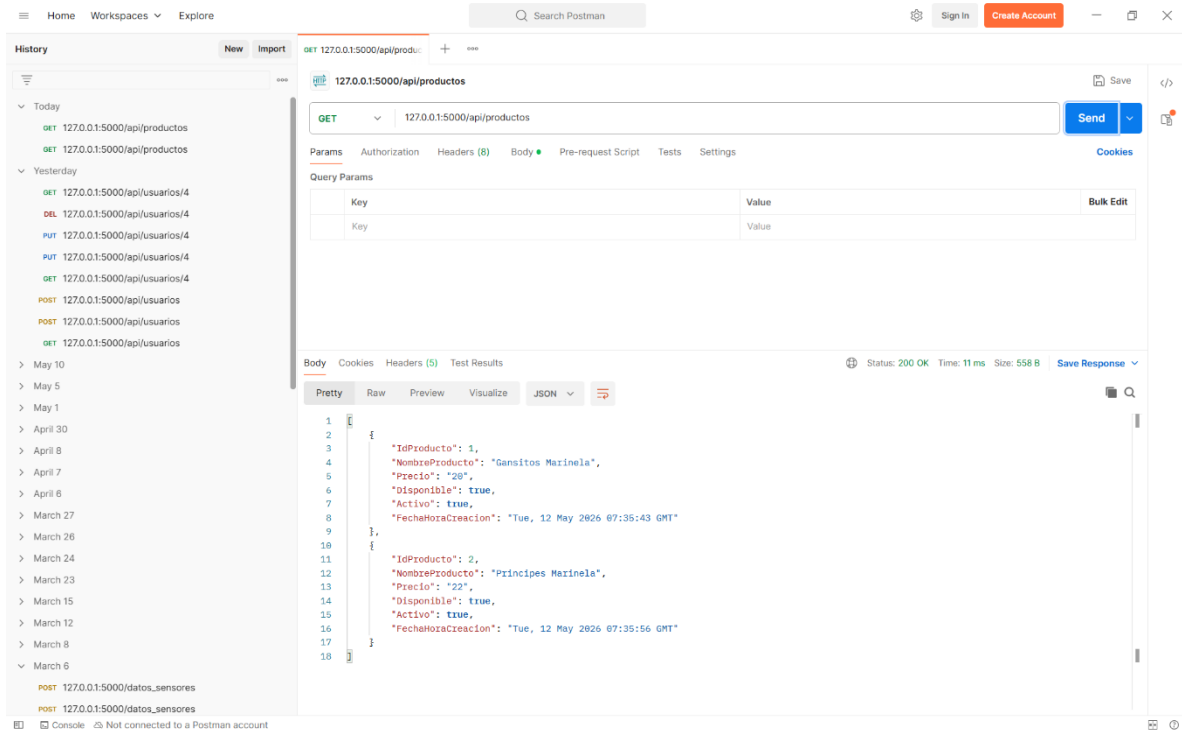


Imagen 38: Solicitud GET exitosa a la entidad Producto

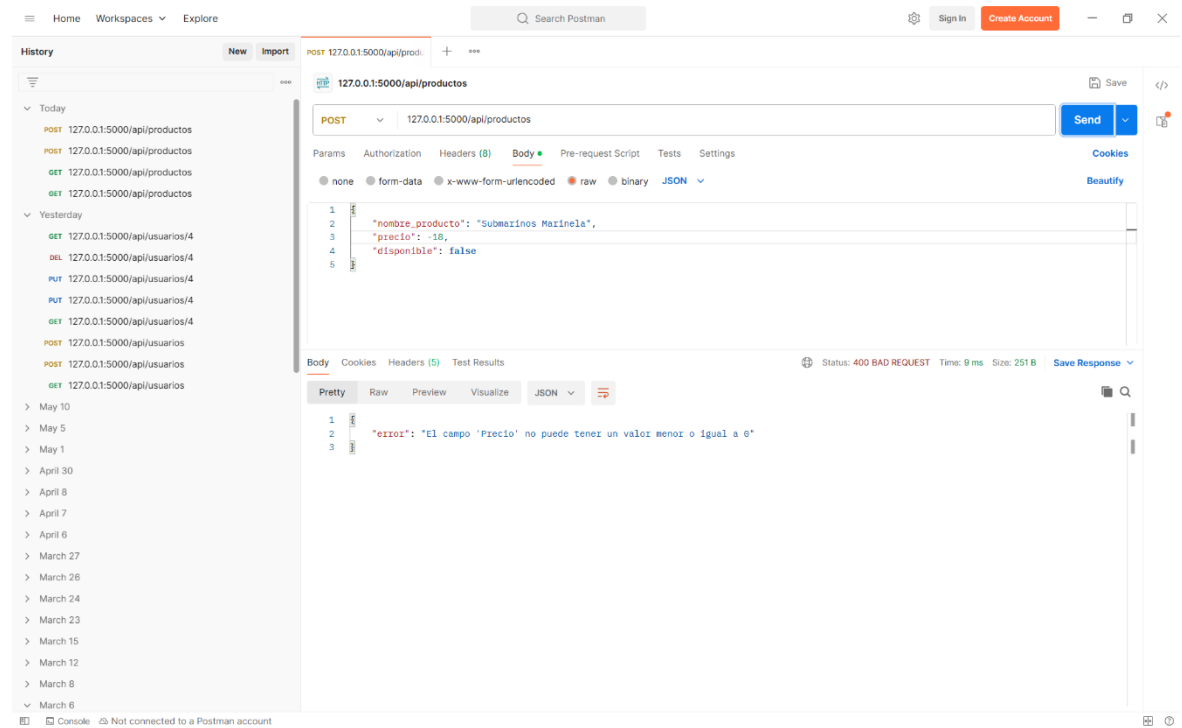


Imagen 39: Solicitud POST err3nea (precio menor a 0)

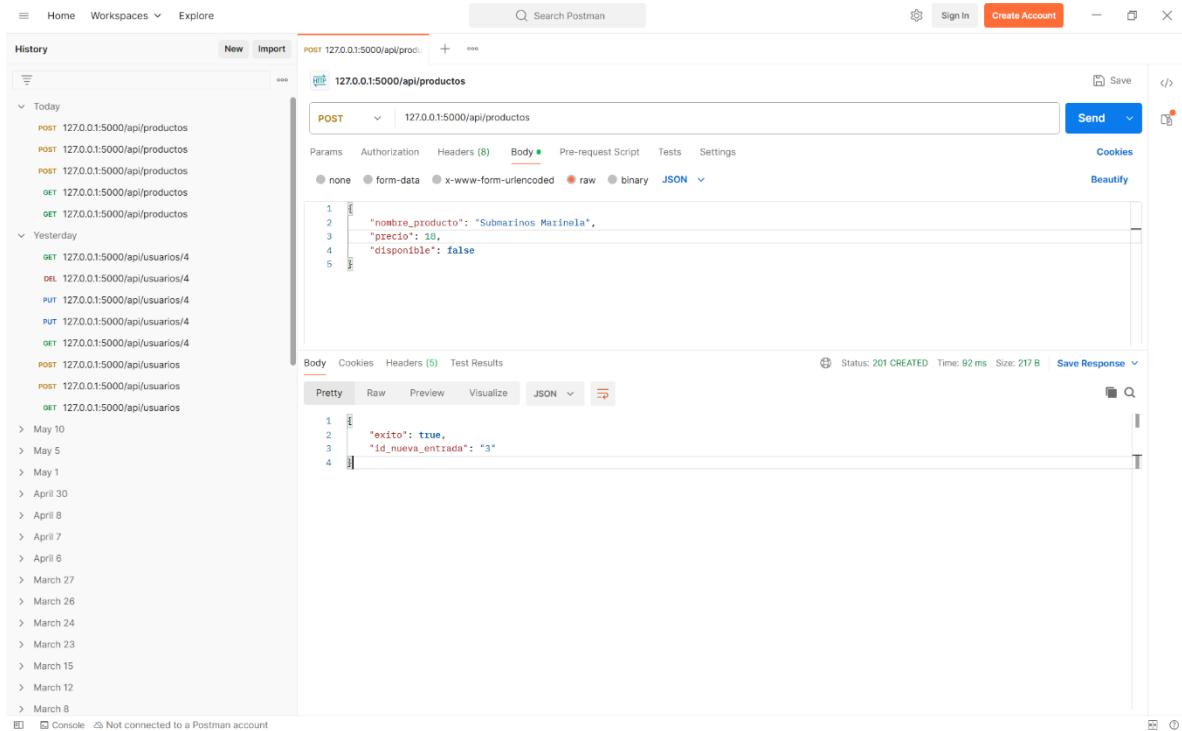


Imagen 40: Solicitud POST exitosa a la entidad Producto

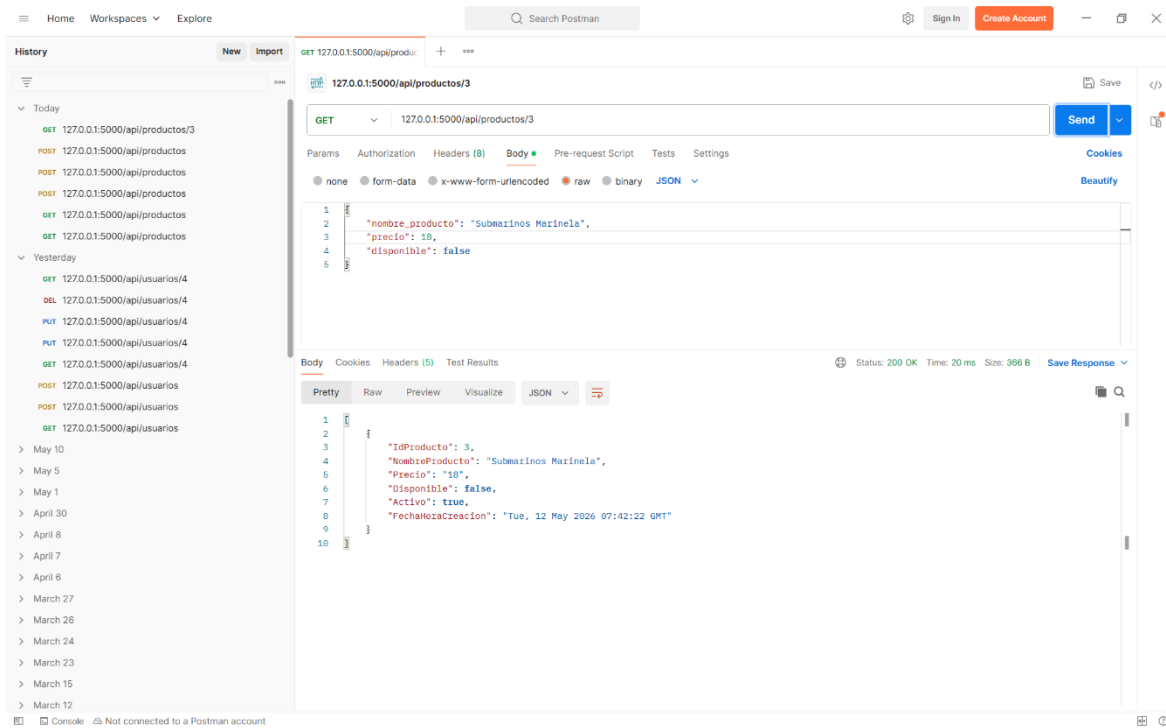


Imagen 41: Solicitud GET por ID exitosa a la entidad Producto

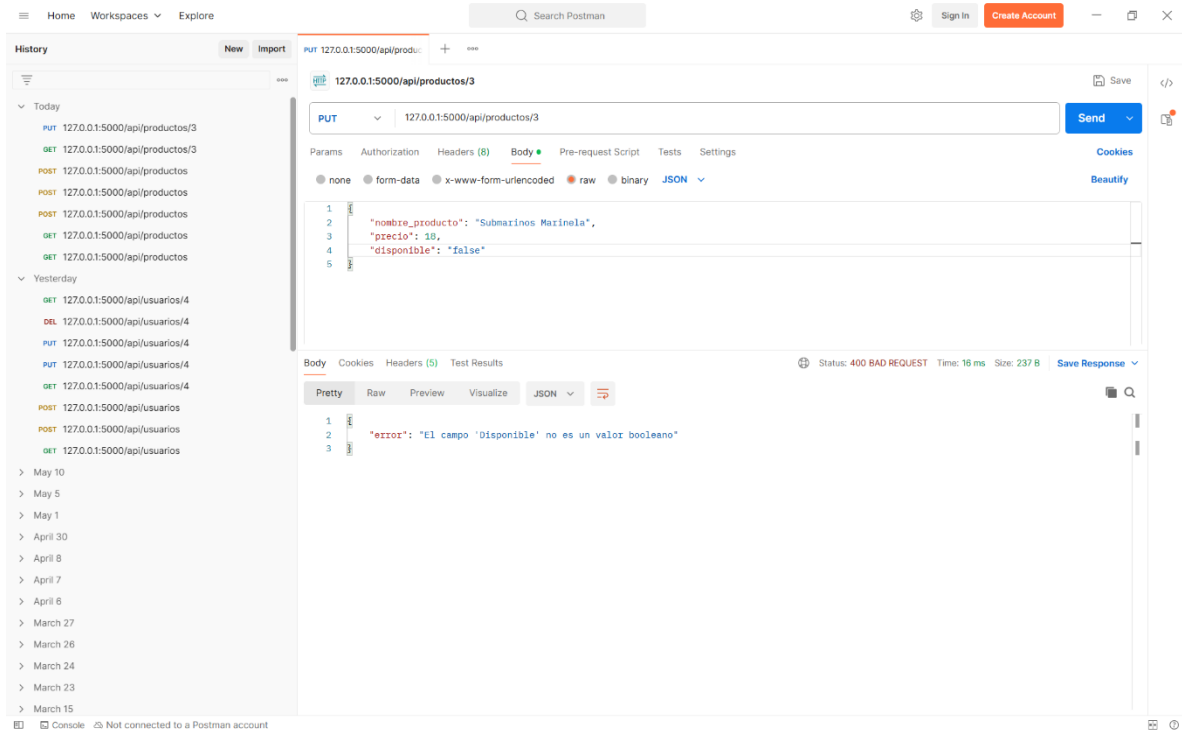


Imagen 42: Solicitud PUT err3nea (entrada disponible de tipo no booleano)

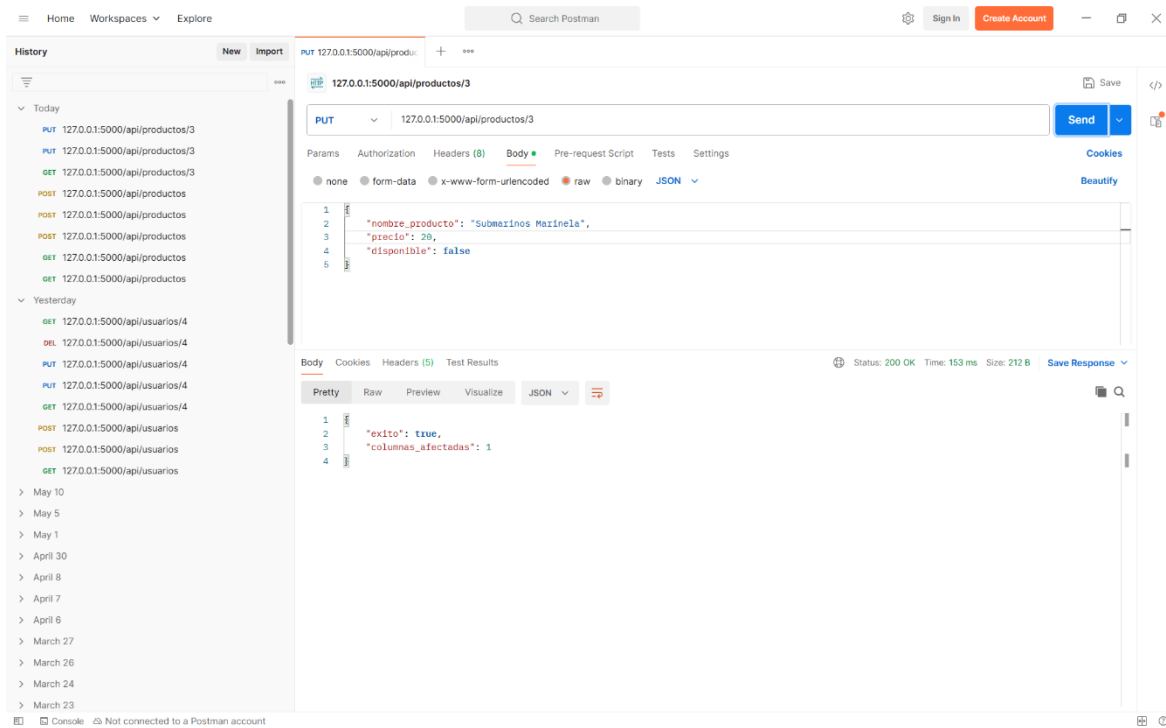


Imagen 43: Solicitud PUT por ID exitosa a la entidad Producto

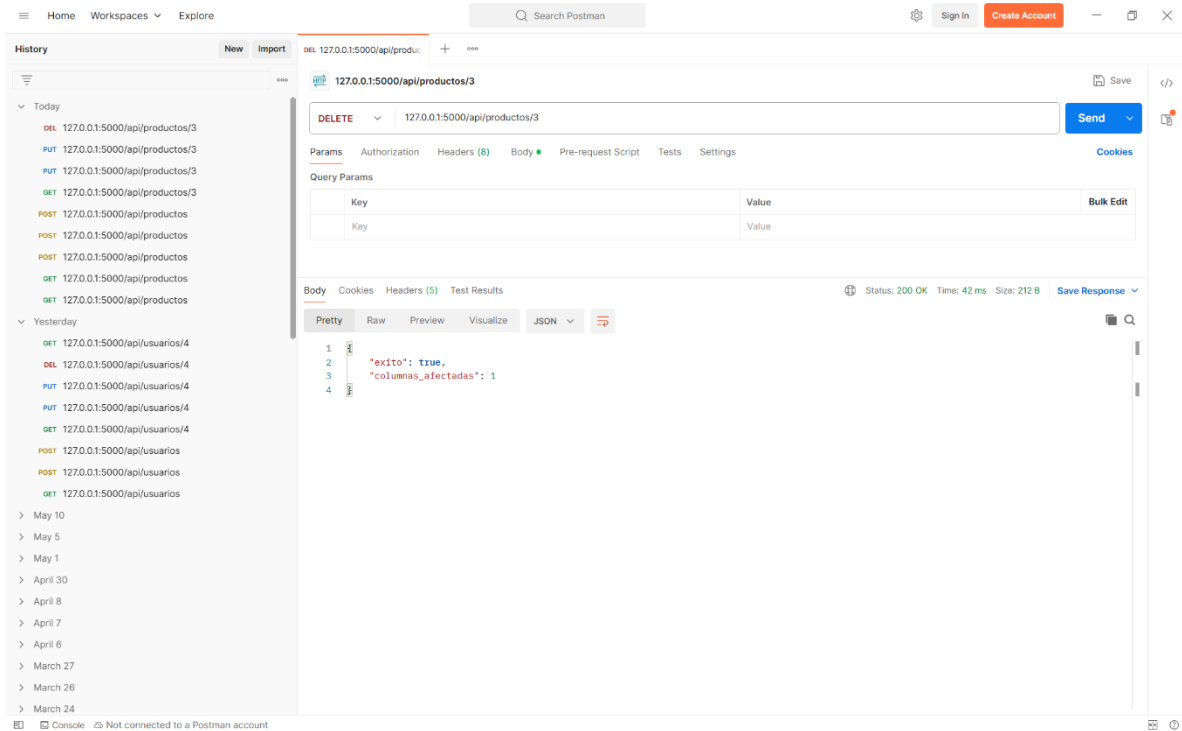


Imagen 44: Solicitud DELETE por ID exitosa a la entidad Producto

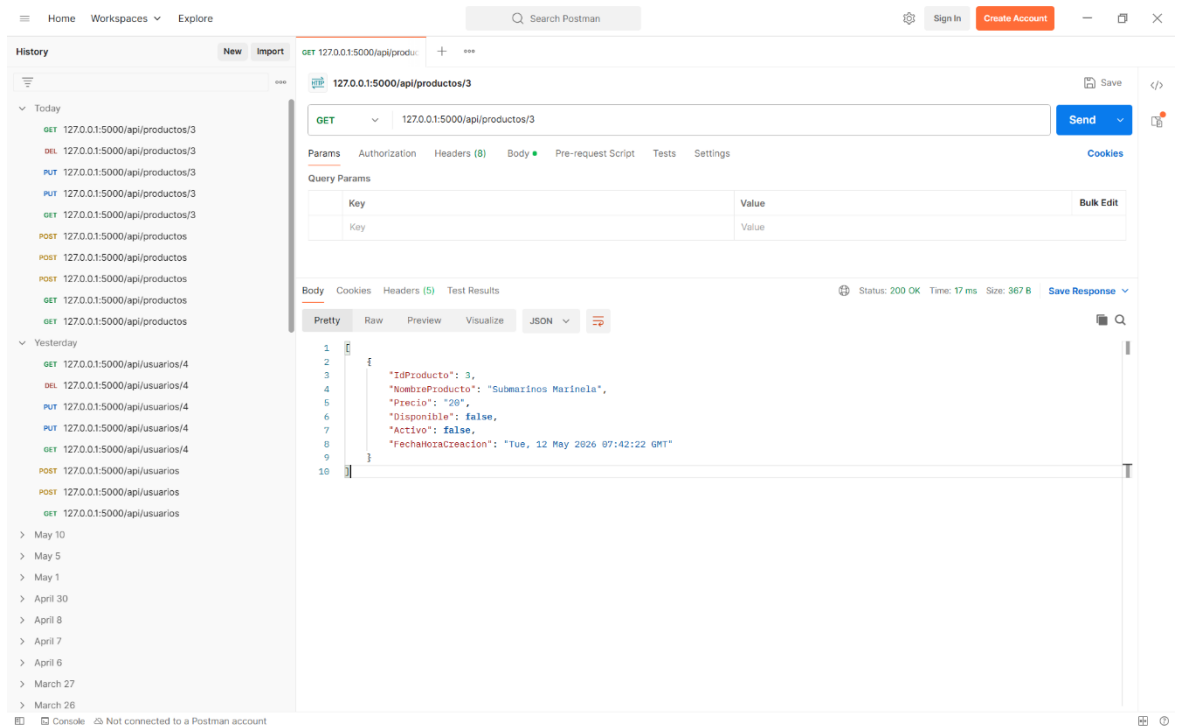


Imagen 45: Verificación de eliminación exitosa (Activo = false)

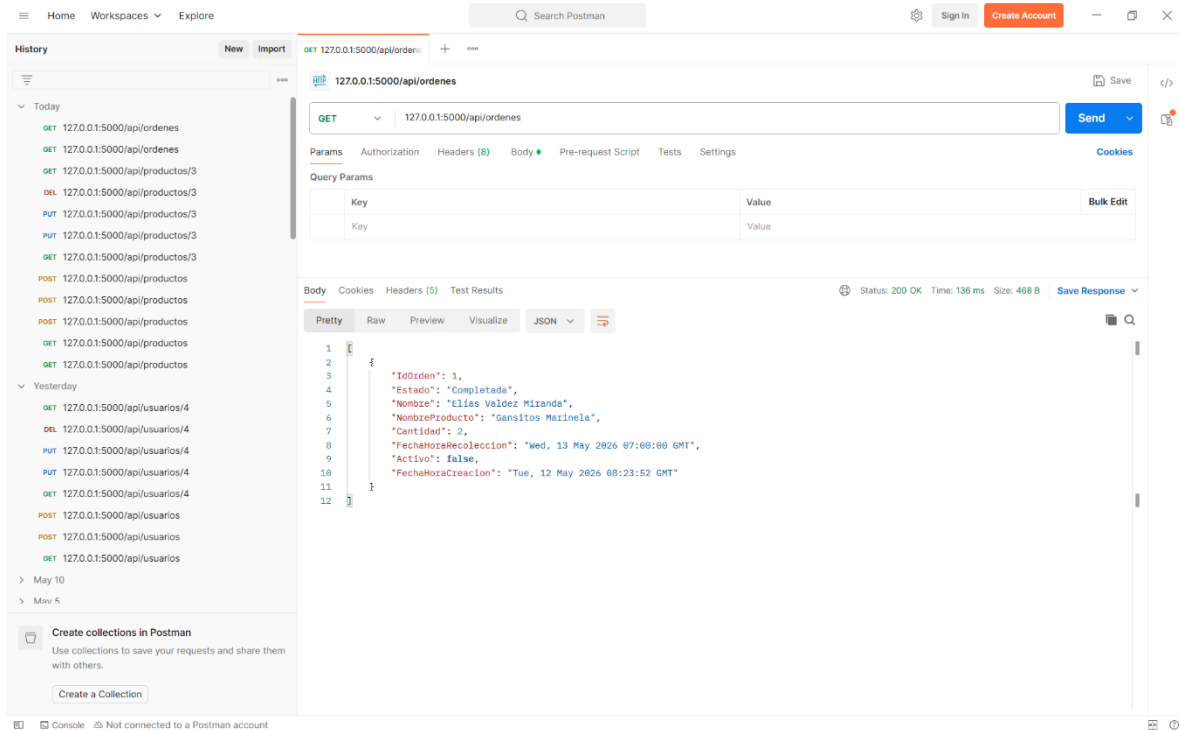


Imagen 46: Solicitud GET exitosa a la entidad Orden

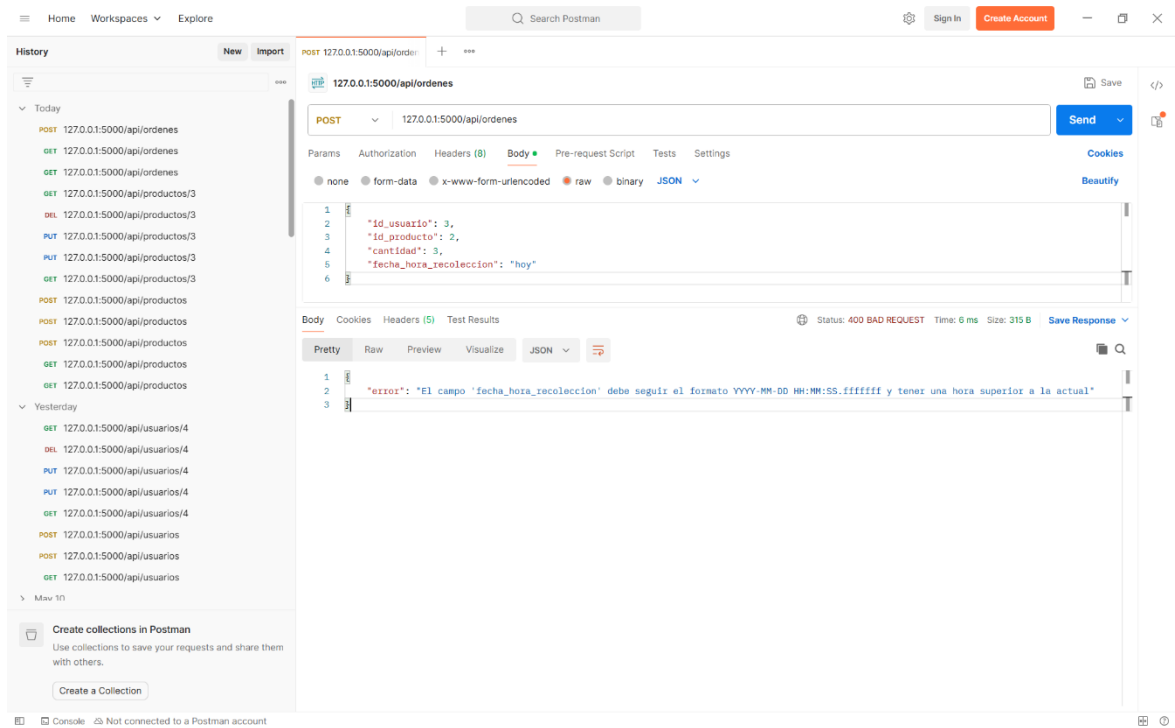


Imagen 47: Solicitud POST errónea (formato de datetime incorrecto)

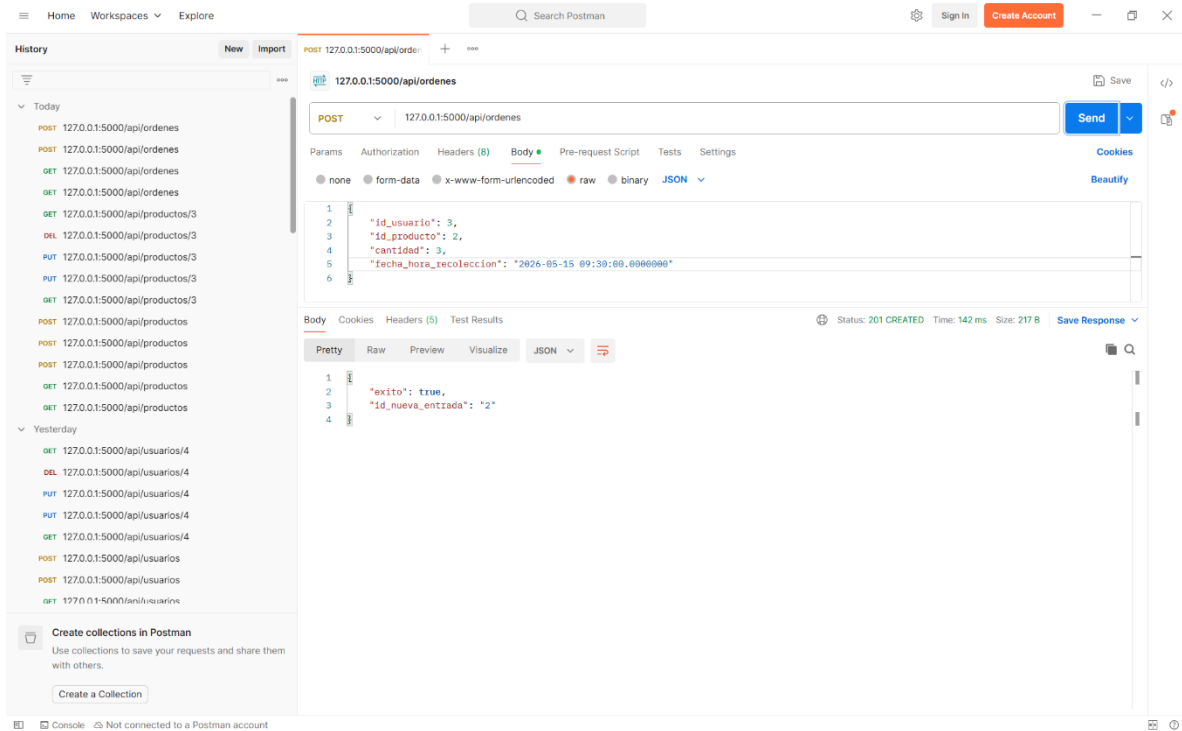


Imagen 48: Solicitud POST exitosa a la entidad Orden

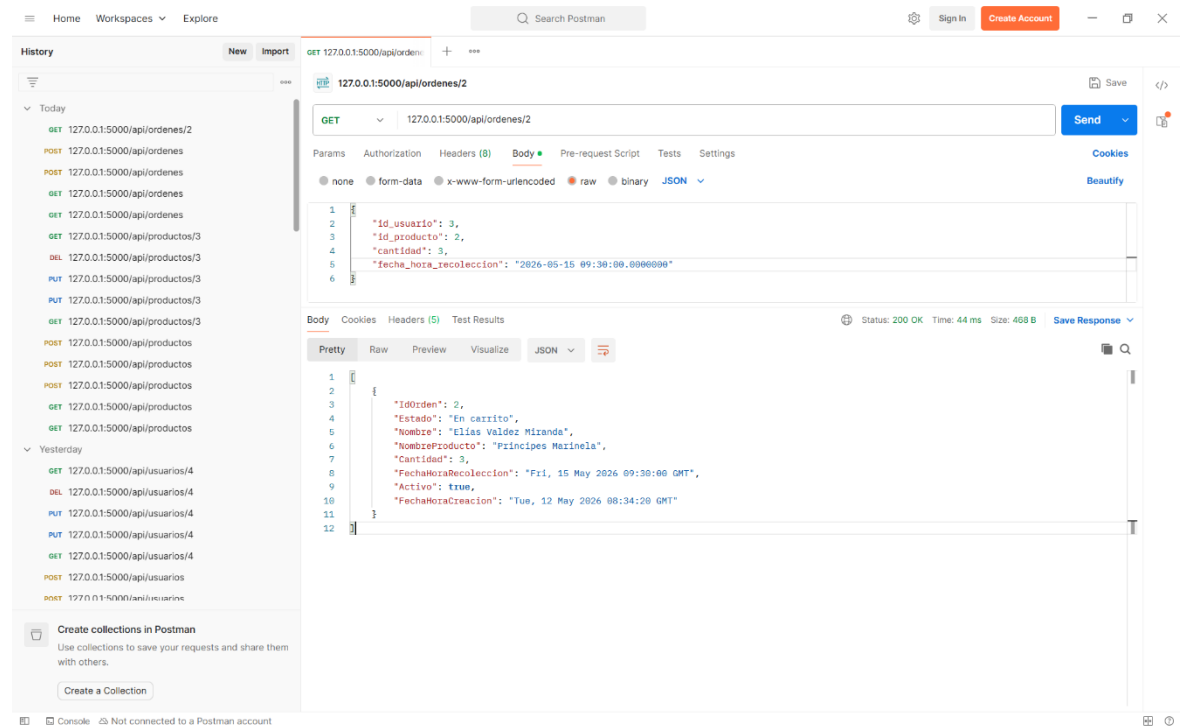


Imagen 49: Solicitud GET por ID exitosa a la entidad Orden

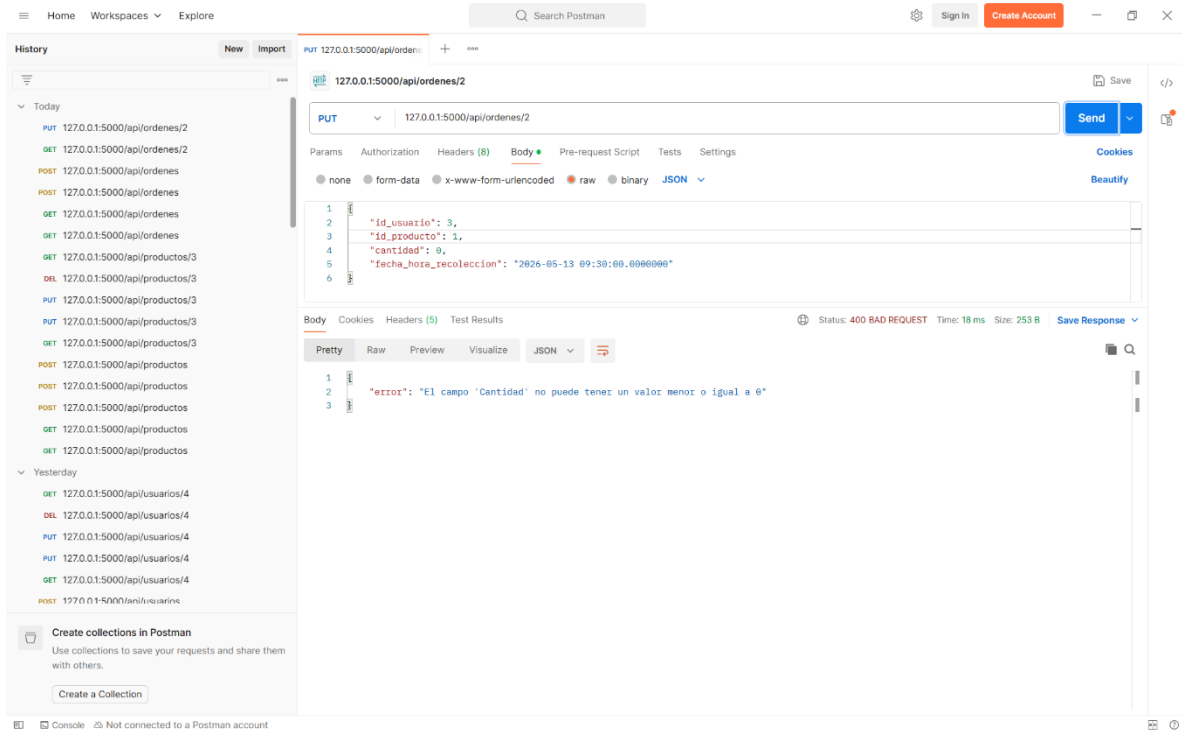


Imagen 50: Solicitud PUT errónea (cantidad con valor 0)

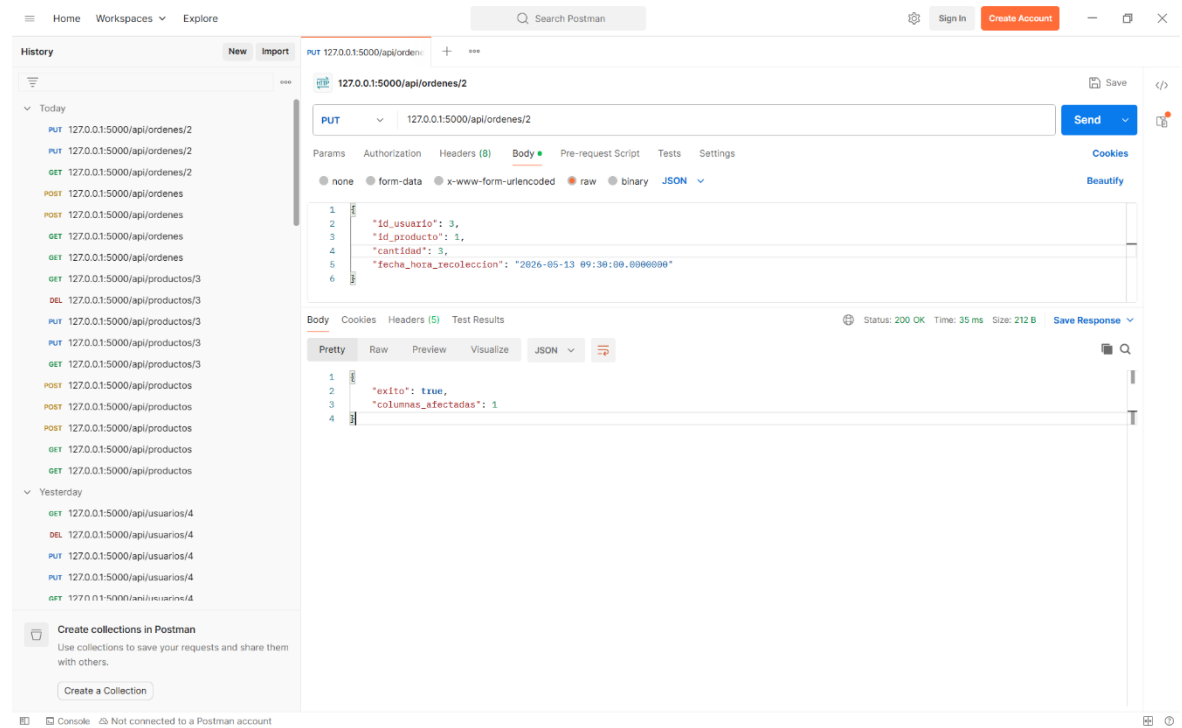


Imagen 51: Solicitud PUT por ID exitosa a la entidad Orden

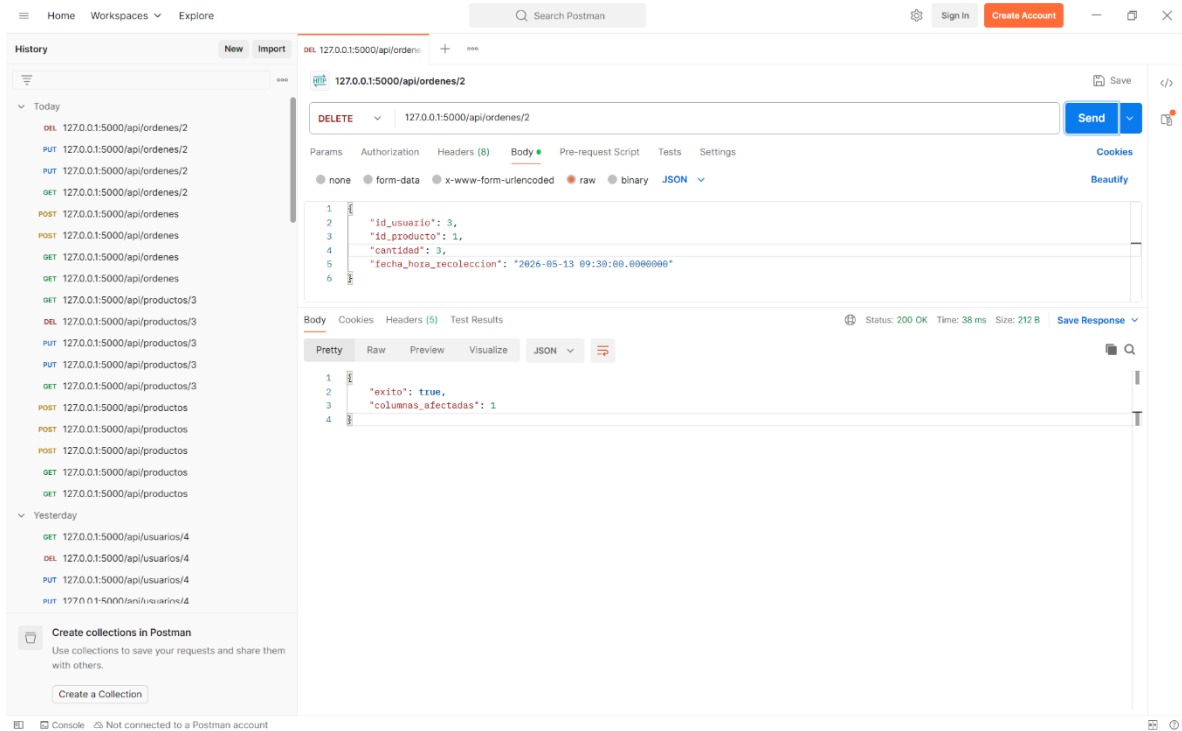


Imagen 52: Solicitud DELETE por ID exitosa a la entidad Orden

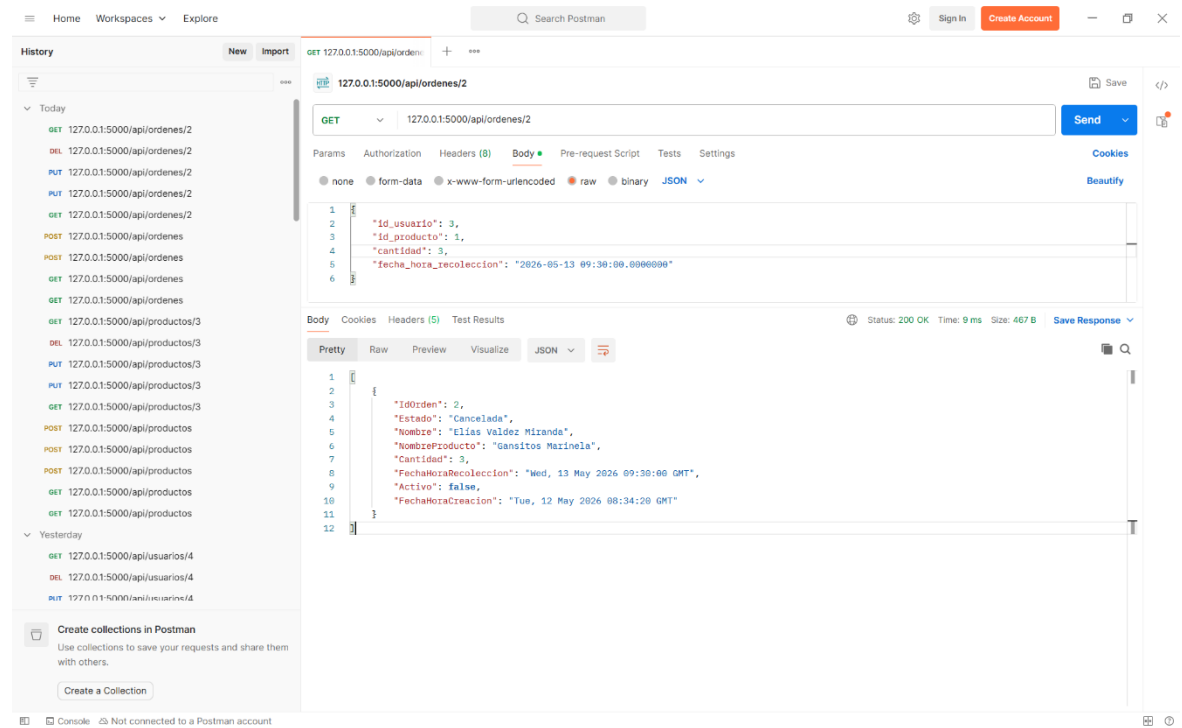


Imagen 53: Verificación de eliminación exitosa (Activo = false)

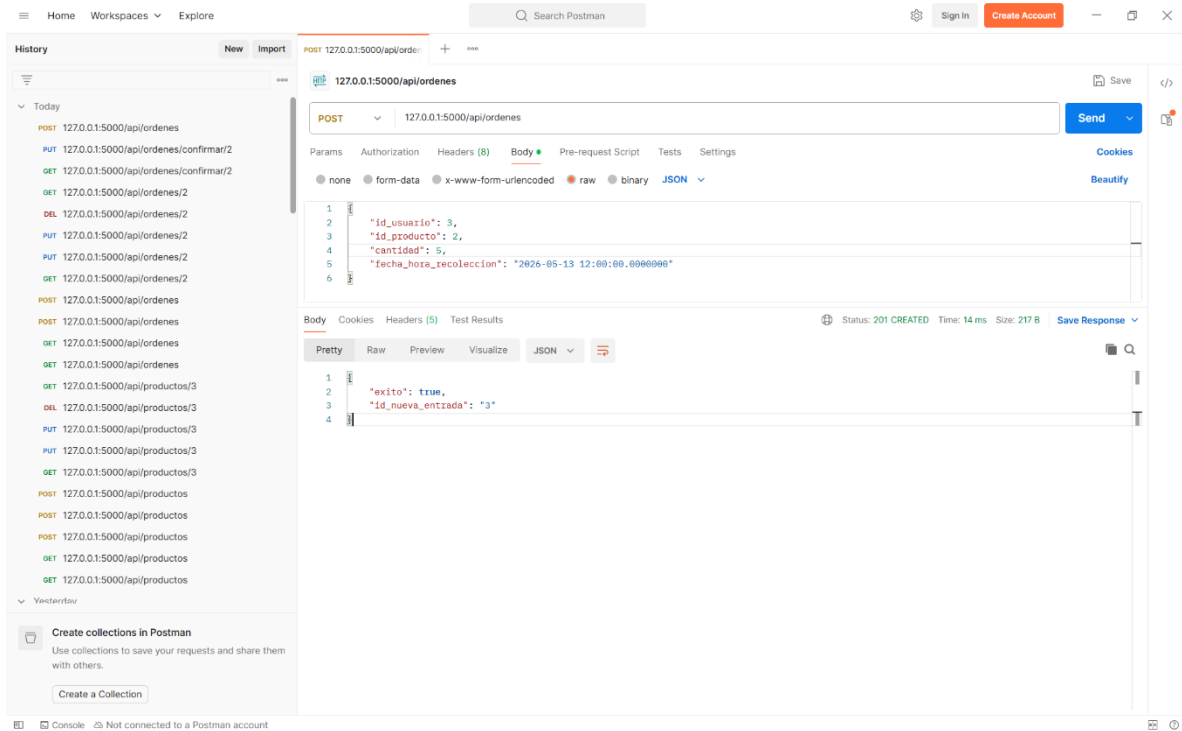


Imagen 54: Creación de nueva orden (para trabajar con rutas posteriores)

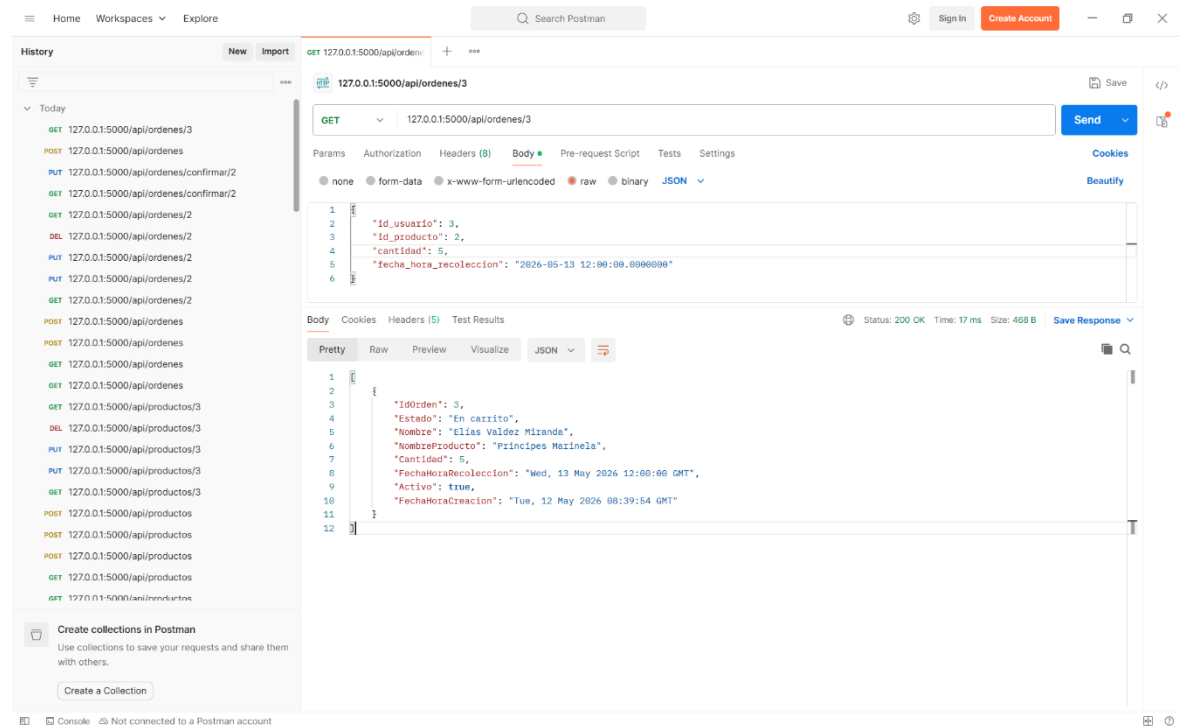


Imagen 55: Información de nueva orden (para trabajar con rutas posteriores)

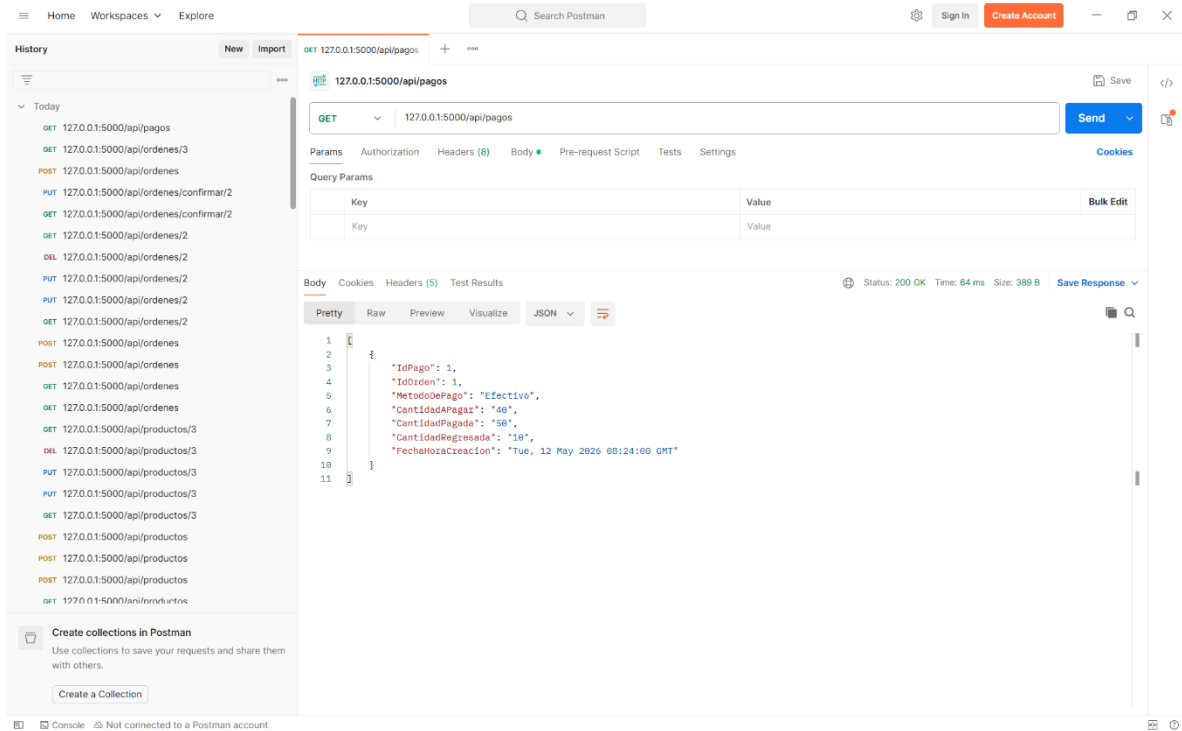


Imagen 56: Solicitud GET exitosa a la entidad Pago

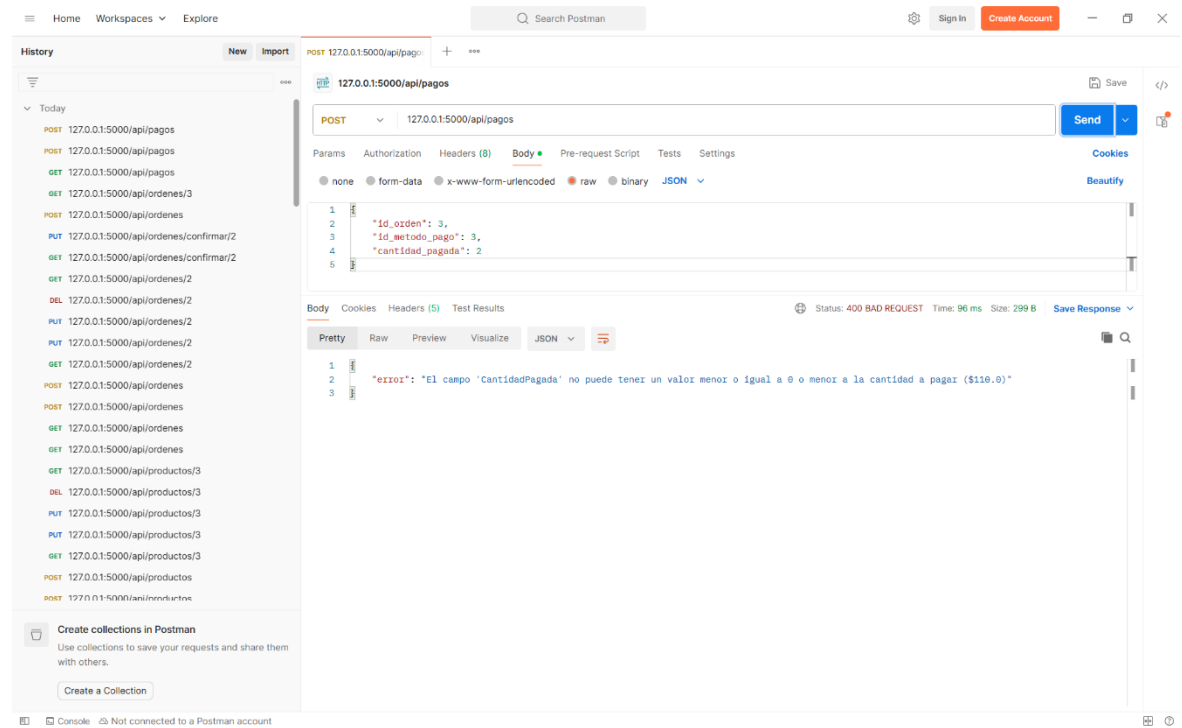


Imagen 57: Solicitud POST err3nea (la cantidad pagada no puede ser menor al total de la orden)

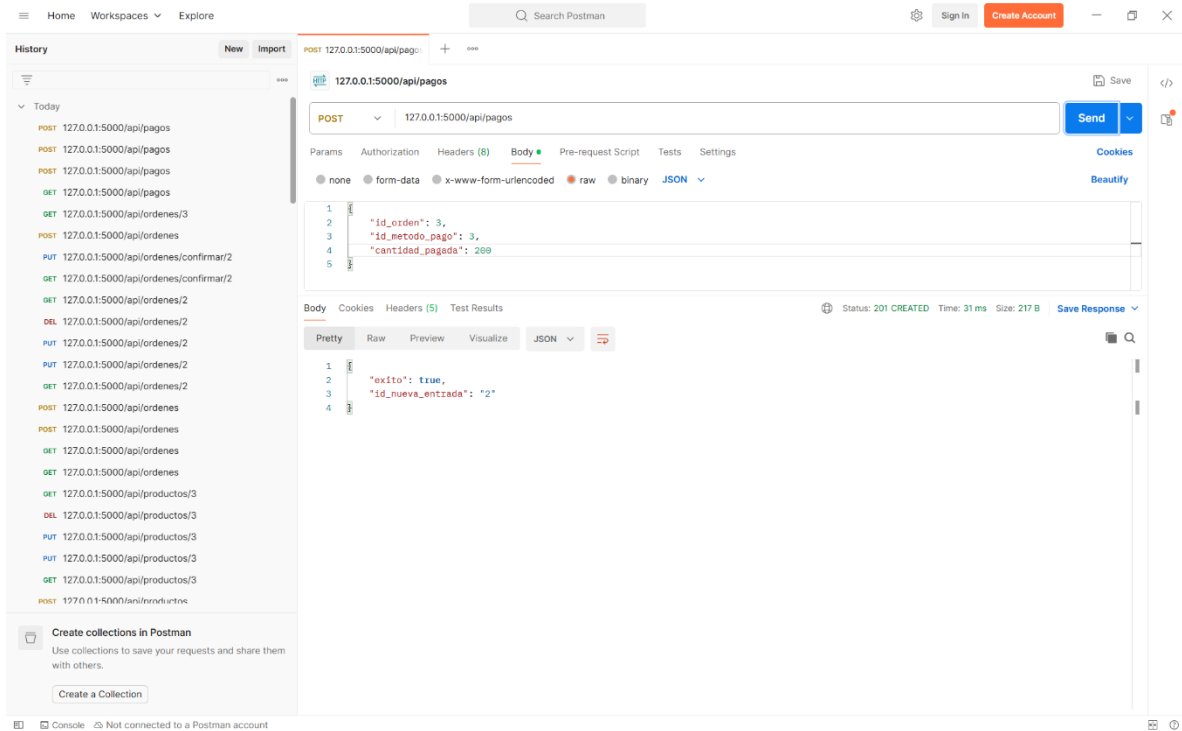


Imagen 58: Solicitud POST exitosa a la entidad Pago

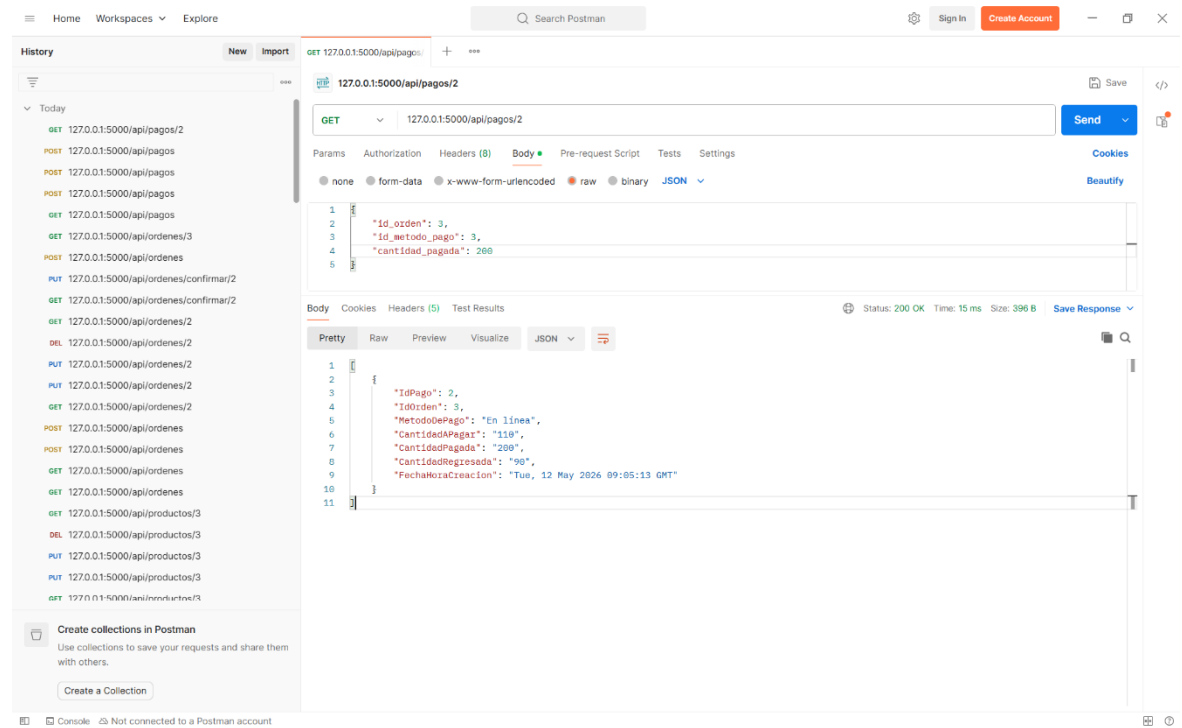


Imagen 59: Solicitud GET por ID exitosa a la entidad Pago

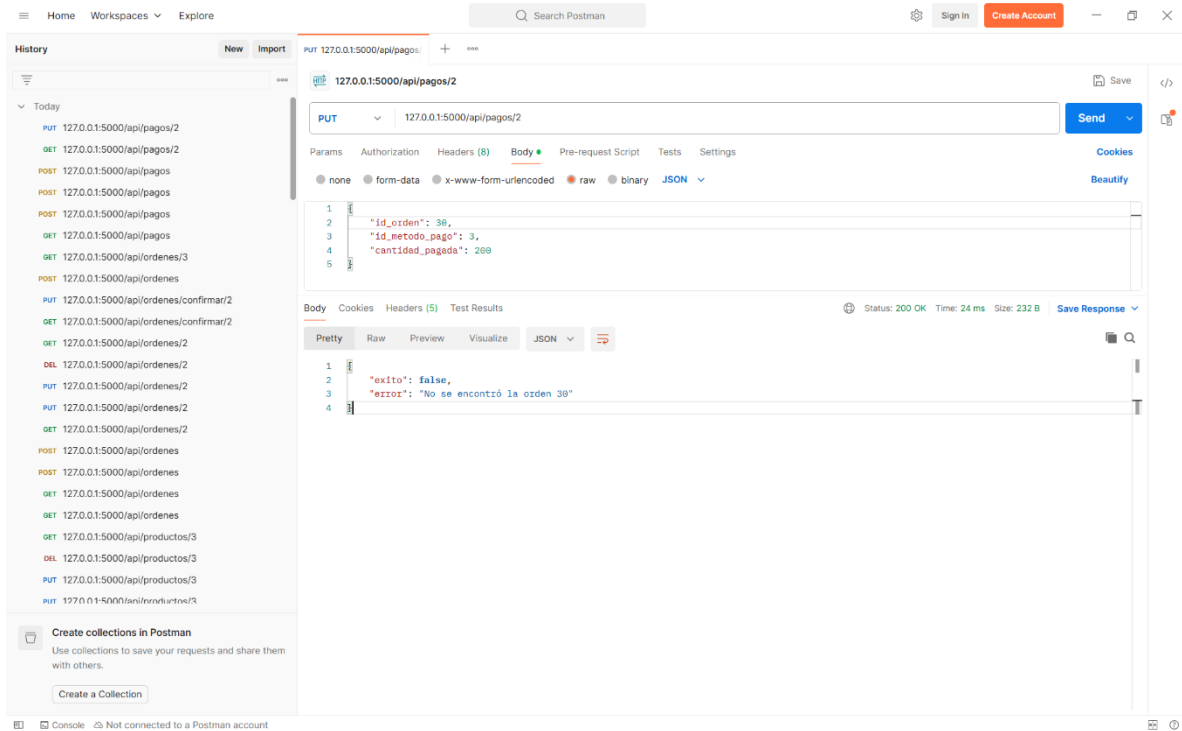


Imagen 60: Solicitud PUT errónea (no se encontró la orden introducida)

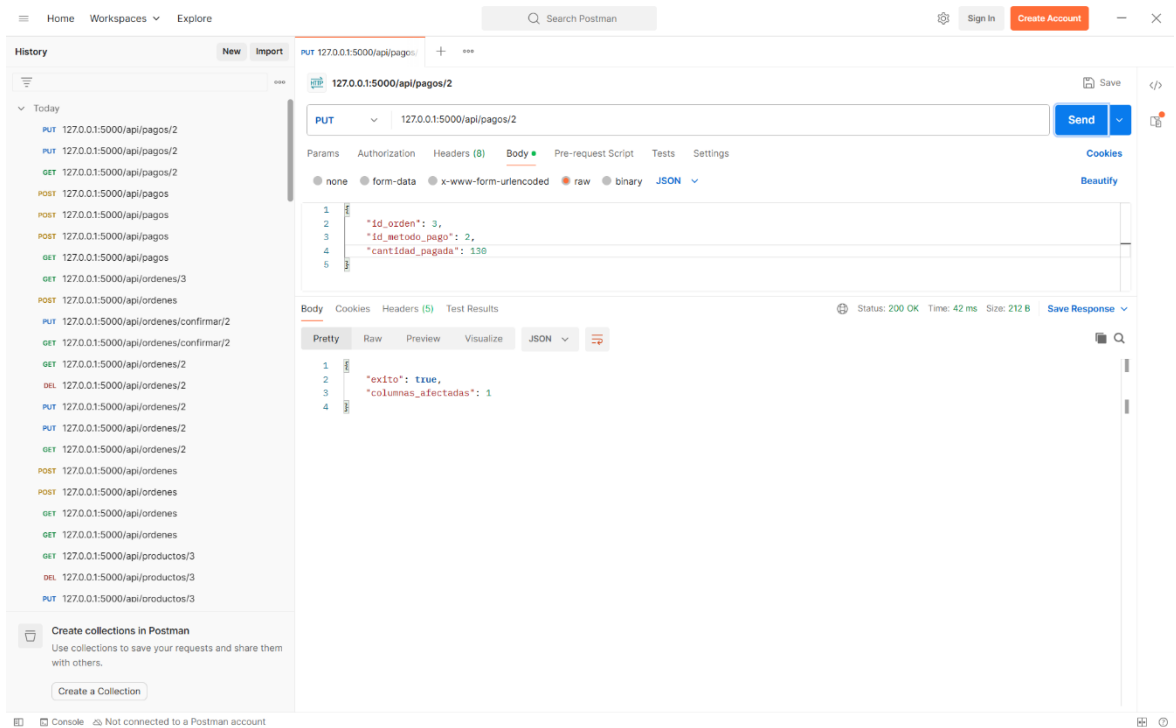


Imagen 61: Solicitud PUT por ID exitosa a la entidad Pago

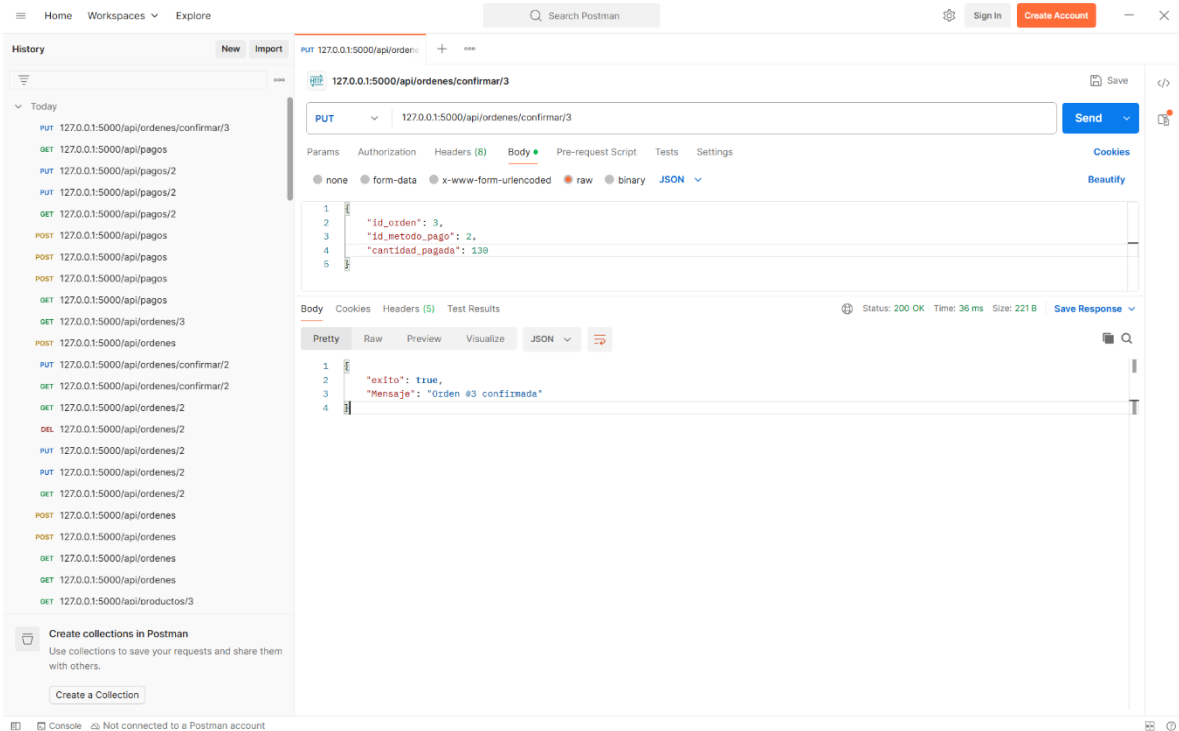


Imagen 62: Confirmación de la orden #3 dada la existencia de un pago

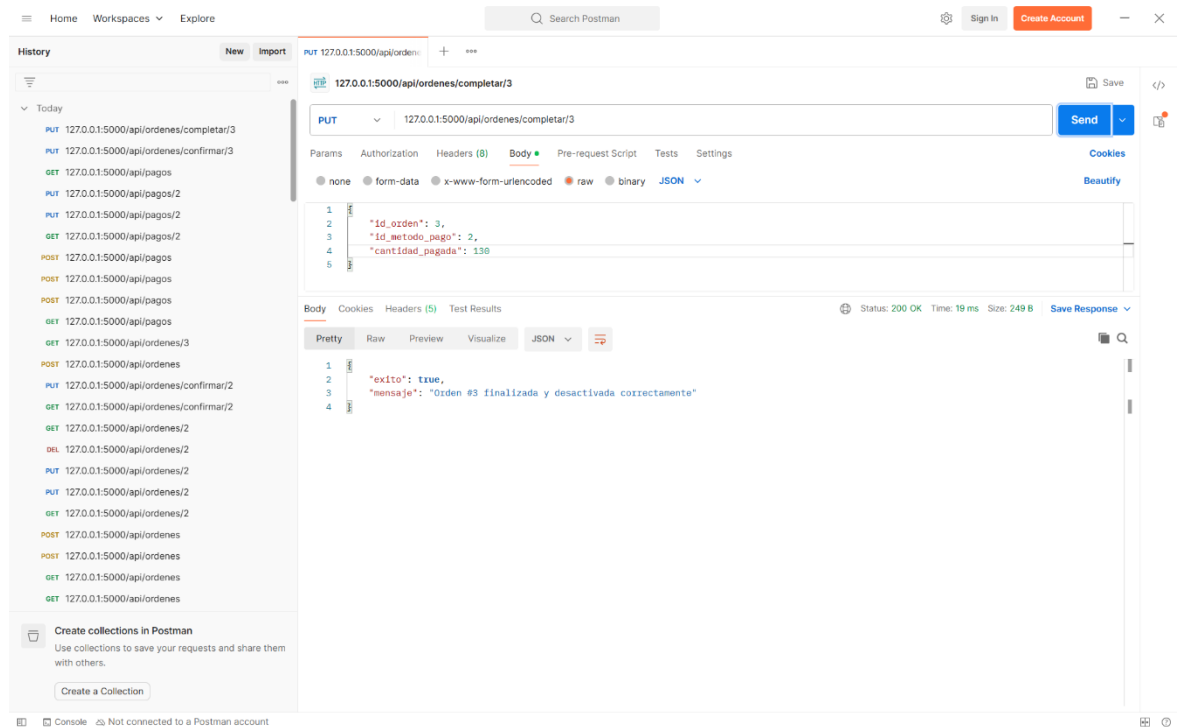


Imagen 63: Finalización de la orden #3 dado el estado “confirmado” de esta

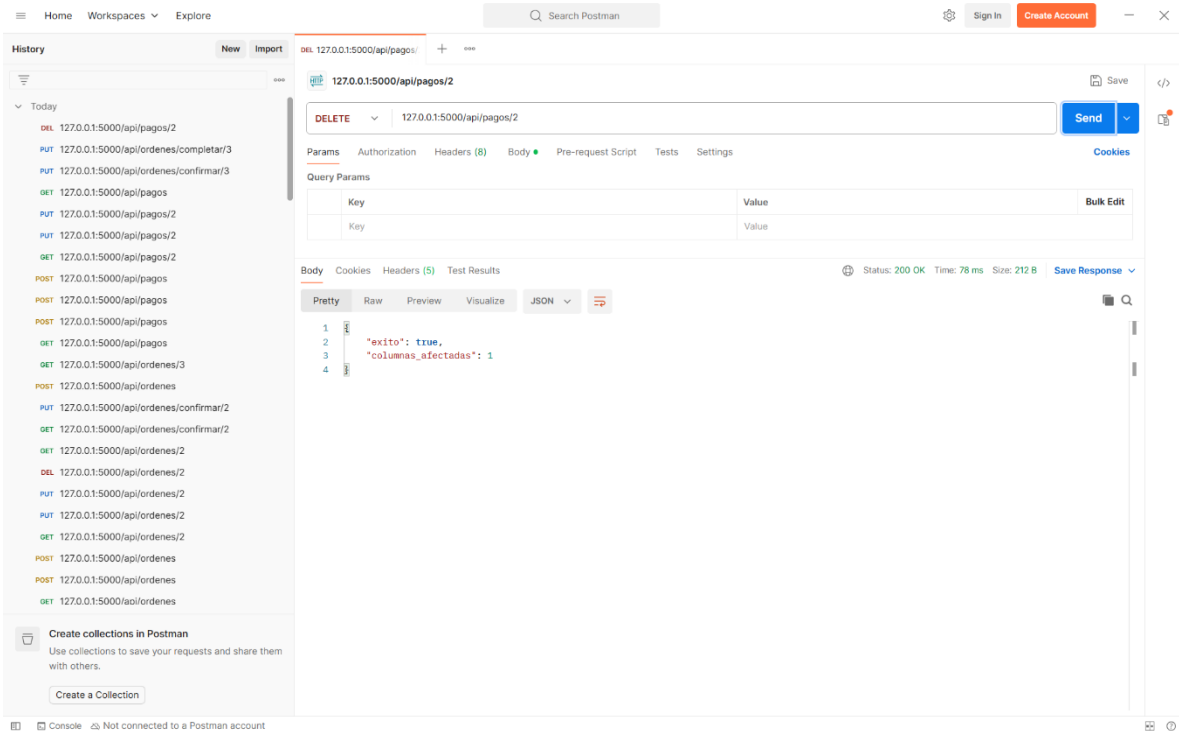


Imagen 64: Solicitud DELETE por ID exitosa a la entidad Pago

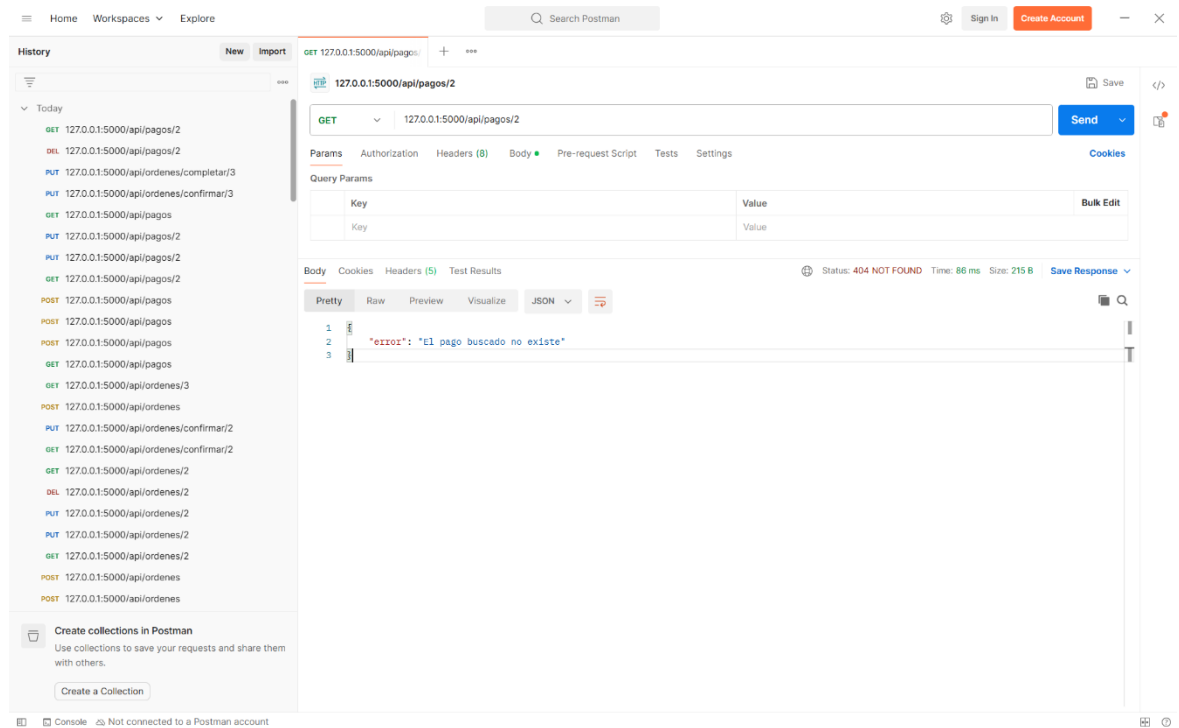


Imagen 65: Verificación de eliminación exitosa (el pago no existe en la base de datos)

Conclusión

Al inicio de este proyecto, la planificación y el diseño de un sistema para la optimización de órdenes en una pequeña tienda presentaba pequeños problemas, considerando únicamente como simplificar el diseño para trabajar con la base de datos y la API REST de forma más sencilla y directa, enfocándome en la implementación de esta junto con un simple frente para el manejo de órdenes. Sin embargo, conforme más avancé el proyecto y empecé a implementar el sistema, encontré que, incluso con las soluciones ofrecidas por Flask, desarrollar un sistema con una API REST supone dificultades que se deben tener presentes antes de empezar el desarrollo.

El concepto más relevante aprendido en el desarrollo de este proyecto fueron los blueprints, los que modularizaron la aplicación de Flask para facilitar la separación de las rutas utilizadas por la aplicación web y la API REST, facilitando la implementación y división de ambas. Otro concepto importante fue el uso del controlador ODBC, el que, aunque funcionaba para la comunicación entre Microsoft SQL Server y Python, requería ciertas consideraciones en el diseño de las consultas y el formato de las respuestas para su uso con el programa, lo que podría haberse facilitado con librerías especializadas (mssql-python) al considerar los dispositivos en los que se trabajaría el programa.

No obstante, a pesar de todos estos problemas, encontré la oportunidad de utilizar los conceptos aprendidos en la materia para el desarrollo de una solución (admisiblemente simplificada y con algunos pobres hábitos) real y efectiva, reiterando la versatilidad de las herramientas utilizadas, incluyendo Python, JSON, Flask, SQL y Postman para probar estas, mejorando mi perfil como desarrollador.

Fuentes de Información

MashaMSFT. (s. f.). Documentación técnica de SQL Server - SQL Server. Microsoft Learn.

<https://learn.microsoft.com/es-es/sql/sql-server/?view=sql-server-ver17>

Modular Applications with Blueprints — Flask Documentation (3.1.x). (s. f.).

<https://flask.palletsprojects.com/en/stable/blueprints/>

GeeksforGeeks. (2025, 23 julio). Flask Blueprints. GeeksforGeeks.

<https://www.geeksforgeeks.org/python/flask-blueprints/>